IUT de Paris - Rives de Seine
Université Paris Cité

Rapport de projet - SAÉ S5.A.01

IQ15Y010 SAÉ Développement avancé

ECO STOCK

Année : 2025-2026

Groupe : 304.1

Réalisé par :

- NGUYEN Phuong
- LIMONTA Evan
- ATCHEKZAI Ismaïl
- BOUZIDI Rayen

Encadrant :

- M. KARINE Ayoub : Référent Technique
- Mme. FRAPPAT Linda : Référent Non-Technique

Date de rendu : Dimanche 11 janvier 2026

Résumé

Chaque année en France, des millions de tonnes de déchets alimentaires sont produites, dont une part importante reste pourtant consommable. Une situation paradoxale alors que de nombreuses personnes en situation de précarité manquent de nourriture au quotidien. Le projet **ECO STOCK** vise à réduire ce gaspillage en proposant une application mobile mettant en relation des commerçants disposant d'invendus et des consommateurs souhaitant acheter à prix réduit. L'application permet la création de comptes (consommateurs et commerçants), la mise en ligne d'offres d'invendus, la consultation et la réservation/achat de produits, ainsi qu'un système de recommandations personnalisées. Une dimension recettes aide également l'utilisateur à valoriser les produits sélectionnés en proposant des idées de plats adaptés. Le projet intègre également une dimension sociale avec des comptes associations donnant accès à des offres réservées, ainsi qu'une fonctionnalité de géolocalisation permettant de proposer des offres situées à proximité de l'utilisateur. Le projet s'appuie sur une application mobile (**React Native**), un back-end (**Node.js/Express**) et une base de données (**PostgreSQL**), avec **authentification sécurisée (JWT)** et paiement en ligne via **Stripe**, les images sont stockées via **cloudinary** pour éviter de les stocker directement dans la base de données. Ce rapport présente le contexte, les objectifs, la méthodologie de travail, les solutions mises en œuvre, les résultats obtenus ainsi que les difficultés rencontrées et les réponses apportées.

Mots-clés **:** gaspillage alimentaire, invendus, application mobile, recommandation, recettes, paiement en ligne

Table des matières

1. Introduction 5
1.1 Accroche et contexte 5
1.2 Problématique 5
1.3 Objectifs généraux du projet 5
2. Analyse du marché et positionnement du projet 7
2.1 Solutions existantes sur le marché 7
2.2 Limites des solutions actuelles 7
2.3 Positionnement et différenciation d'ECO STOCK 7
3. Méthodologie 9
3.1 Gestion de projet : La méthode Agile Scrum 9
3.2 Outils de collaboration et technique 10
3.3 Déroulement des sprints 11
4. Cahier des charges et besoins 12
4.1 Besoins fonctionnels 12
4.2 Contraintes techniques et non fonctionnelles 13
4.3 Critères de validation 14
5. Conception et solutions mises en œuvre 16
5.1 Architecture générale (mobile ↔ API ↔ base de données) 16
5.2 Conception de la base de données (description textuelle des liens) 22
5.3 Sécurité : authentification, sessions, contrôles d'accès, protection des secrets 25
5.4 Gestion des images avec Cloudinary (éviter le base64 en BDD) 26
5.5 Géolocalisation et calcul des distances/temps d'accès 27
5.6 Logique "DLC" + fenêtre de retrait (anti-gaspillage réaliste) 27
5.7 Recherche intelligente et tolérante aux fautes (ingrédients + description) 28
5.8 Lots (panier surprise) vs produit simple : logique et impact recettes 29
5.9 Recettes : lien entre ce que l'utilisateur achète et comment l'utiliser 30
5.10 Paiement en ligne et finalisation de commande 30
5.11 Associations : offres réservées et accès différencié 31
5.12 Frontend : organisation, navigation, séparation UI / logique 31
5.13 Pagination et optimisation des performances 32
5.14 Gestion avancée du stock et prévention des conflits 32
5.15 Calcul et affichage des réductions 33
5.16 Synchronisation et persistance du panier 33

5.17 Index et optimisations de la base de données 34
5.18 Triggers et automatisations PostgreSQL 34
6. Tests et validation 35
6.1 Tests continus 35
6.2 Validation des Performances 40
7. Résultats obtenus et analyse 44
7.1 Fonctionnalités implémentées et validation des objectifs 44
7.2 Performances techniques et optimisations 45
7.3 Qualité du code et maintenabilité 47
7.4 Sécurité et fiabilité 47
7.5 Différenciation par rapport aux solutions existantes 47
7.6 Limites et axes d'amélioration identifiés 48
7.7 Bilan global 49
8. Difficultés rencontrées et solutions apportées 50
9. Conclusion et perspectives 58
9.1 Bilan du projet 58
9.2 Ce qu'on a appris 58
9.3 Les difficultés rencontrées 58
9.4 Ce qui pourrait être amélioré 59
9.5 Mot de la fin 59
Synthèses individuelles 60
Bibliographie / Webographie 62

1. Introduction

1.1 Accroche et contexte

Chaque année, le gaspillage alimentaire représente un enjeu environnemental, économique et social majeur. En France, une quantité significative de produits encore consommables est jetée ou détruite, notamment dans le secteur de la distribution et du commerce de proximité. Paradoxalement, cette perte coexiste avec une demande croissante pour des produits alimentaires à prix accessibles et une volonté accrue des consommateurs d'adopter des comportements plus responsables.
Face à ce constat, le numérique apparaît comme un levier pertinent pour faciliter la mise en relation entre les acteurs concernés et optimiser la valorisation des invendus. Les applications mobiles, largement utilisées au quotidien, offrent un support adapté pour proposer des solutions simples, rapides et accessibles à un large public.
C'est dans ce contexte que s'inscrit le projet ECO STOCK, développé dans le cadre de la SAÉ S5.A.01. Ce projet vise à concevoir une application mobile permettant de mettre en relation des commerçants disposant d'invendus alimentaires et des consommateurs souhaitant acheter ces produits à prix réduit, tout en contribuant à la réduction du gaspillage alimentaire. La possibilité d'observer directement des recettes à partir de l'application peut encourager l'acheteur à acheter des produits, ce qui permet donc de réduire le gaspillage.

1.2 Problématique

La réalisation de ce projet s'articule autour de la problématique suivante :

Comment concevoir une application mobile fiable et sécurisée permettant de valoriser les invendus alimentaires des commerçants, tout en offrant une expérience utilisateur simple, transparente et adaptée aux besoins des consommateurs ?

Cette problématique soulève plusieurs enjeux techniques et fonctionnels : la gestion sécurisée des comptes utilisateurs, la publication et la consultation d'offres en temps réel, la prise en compte des contraintes liées aux stocks et aux dates limites de consommation, ainsi que l'intégration d'un système de paiement en ligne fiable. Elle implique également une réflexion sur l'ergonomie de l'application et sur la cohérence globale du système afin d'assurer une utilisation fluide et efficace.

1.3 Objectifs généraux du projet

Afin de répondre à cette problématique, le projet ECO STOCK poursuit les objectifs suivants :

- Concevoir une application mobile permettant aux commerçants de publier et gérer leurs invendus alimentaires.

- Offrir aux consommateurs une interface simple pour consulter, réserver ou acheter des produits à prix réduit.

- Assurer la sécurité des données et des transactions grâce à des mécanismes d'authentification et de paiement adaptés.

- Proposer des fonctionnalités complémentaires, telles que des recommandations et des idées de recettes, afin de favoriser une consommation responsable et limiter le gaspillage à domicile.

La suite de ce rapport détaille le cahier des charges établi, la méthodologie de travail adoptée, les solutions techniques mises en œuvre, les résultats obtenus ainsi que les difficultés rencontrées et les enseignements tirés de cette expérience.

2. Analyse du marché et positionnement du projet

2.1 Solutions existantes sur le marché

Plusieurs solutions numériques existent aujourd'hui pour lutter contre le gaspillage alimentaire en facilitant la revente d'invendus. Parmi les plus connues, on peut citer **Too Good To Go** et **Phenix** qui proposent des applications mobiles mettant en relation des commerçants et des consommateurs.

Ces plateformes reposent principalement sur un modèle de paniers surprise, vendus à prix réduit que l'utilisateur réserve sans connaître précisément le contenu. Ce fonctionnement permet une réduction efficace des invendus mais il limite la transparence pour le consommateur qui ne peut ni choisir les produits ni adapter ses achats à ses besoins réels.

Par ailleurs, bien que certaines initiatives collaborent avec des associations. Cet aspect reste souvent secondaire dans l'interface utilisateur et peu visible pour le grand public.

2.2 Limites des solutions actuelles

L'analyse de ces solutions met en évidence plusieurs limites :

- Effet de surprise lié au principe du panier surprise, qui peut ne pas convenir à tous.
- Une faible personnalisation de l'expérience utilisateur.
- Une visibilité limitée des produits disponibles avant l'achat.
- Peu de fonctionnalités visant à accompagner le consommateur dans l'utilisation des produits récupérés.
- Intégration souvent secondaire ou peu lisible de la dimension associative.
- Aucune incitation qui favorise l'achat comme des recommandations de recettes ou offres.

Ces limites peuvent freiner l'adhésion de certains utilisateurs, notamment ceux souhaitant maîtriser davantage leurs achats ou s'inscrire dans une démarche de consommation responsable sur le long terme.

2.3 Positionnement et différenciation d'ECO STOCK

Afin de répondre aux limites identifiées, le projet ECO STOCK se distingue des solutions existantes par une approche plus transparente, inclusive et orientée utilisateur. Il ne se limite pas à une simple place de marché d'invendus mais accompagne l'utilisateur de l'achat jusqu'à

la consommation, notamment grâce à des fonctionnalités d'aide à l'utilisation des produits, tout en intégrant une dimension sociale.

Contrairement au principe du panier surprise, ECO STOCK permet aux consommateurs de visualiser précisément les produits proposés, leurs caractéristiques (prix, quantité, date limite de consommation, la description, panier en lot et sa catégorie) et de sélectionner ceux qui correspondent réellement à leurs besoins. Cette transparence favorise une consommation plus réfléchie et limite le gaspillage.

Un système de géolocalisation est inclus dans le projet permettant de proposer aux utilisateurs des offres situés à proximité de leur position que ça soit à pied ou en transport. Cette fonctionnalité facilité l'accès aux invendus proposés par les commerces locaux et limite les déplacements inutiles.

L'application intègre également une fonctionnalité associative, en proposant des comptes dédiés aux associations. Ces dernières peuvent accéder à des offres spécifiques ou réservées, facilitant ainsi la redistribution des invendus auprès des publics dans le besoin et renforçant l'impact social du projet.

Par ailleurs, ECO STOCK propose des fonctionnalités complémentaires absentes sur les plateformes concurrentes, telles que :

- un système de recommandations personnalisées, visant à orienter l'utilisateur vers des produits en cohérence avec ses habitudes.
- un onglet "recettes", permettant de suggérer des idées de plats à partir des produits achetés, afin d'accompagner l'utilisateur dans leur valorisation et de réduire le gaspillage à domicile.
- une recherche intelligente par ingrédients, tolérant des variations d'écriture (accents, pluriels, fautes simples) et proposant des correspondances pour améliorer la pertinence des résultats.
- une expérience utilisateur simplifiée, reposant sur une interface claire et intuitive, facilitant l'adoption de l'application par un large public.

Enfin, l'application intègre des éléments de sensibilisation à la réduction du gaspillage alimentaire, notamment par l'affichage de messages courts lors de certaines phases de chargement comme celui pour se connecter. Ces messages participent à renforcer l'identité écologique du projet et à maintenir l'utilisateur conscient des enjeux environnementaux liés à la consommation.

Ainsi, ECO STOCK ne se limite pas à la revente d'invendus mais s'inscrit dans une démarche globale visant à encourager une consommation responsable, durable et socialement engagée.

3. Méthodologie

Pour mener à bien le développement de l'application EcoStock, nous avons choisi d'utiliser la méthode agile Scrum car elle permet d'être flexible et favorise la communication.

3.1 Gestion de projet : La méthode Agile Scrum

Cette méthodologie de gestion de projet repose sur une communication claire grâce aux différentes cérémonies, notamment les "dailys" qui sont des réunions quotidiennes courtes faites en début de journée qui permettent d'échanger sur l'avancée du projet en suivant trois thèmes principaux : le travail effectué la veille, ce que l'on prévoit de faire aujourd'hui et les principaux problèmes rencontrés. Il y a également le "sprint planning" qui est une réunion lors de laquelle on prévoit les tâches, on leur accorde une note de complexité, une "user story" qui est un scénario d'utilisation de cette tâche et pour finir une "definition of done" qui sont des critères de validité de cette tâche. Ces tâches sont ensuite séparées en différents sprints en fonction de leur complexité et de leurs priorités. Pour finir, il y a également des réunions rétrospectives qui permettent de revenir sur un sprint une fois qu'il est achevé, ces réunions permettent de faire un bilan du sprint.

Nous avons choisi d'utiliser la méthode Scrum car c'est la méthode que l'on connaissait le mieux, et que c'est une des méthodes les plus utilisées dans le monde professionnel. Après s'être occupés de la rédaction du cahier des charges, nous avons donc commencé le projet en utilisant la méthode Scrum avec l'outil Trello afin d'organiser notre projet. Nous avons commencé par le sprint planning et la création du backlog, nous avons créé des tickets pour les différentes fonctionnalités que nous avions ciblées pour l'application. Pour chaque ticket, nous avons défini une user story, une complexité basée sur la suite de Fibonacci (1,2, 3, 5, 8), et une "definition of done". Nous les avons ensuite triés par ordre d'importance. Ainsi, nous avons réparti les tâches en sprints en prenant en compte les critères de complexité et de priorité, chaque sprint était prévu pour une durée de deux semaines.

Cette méthodologie nous a permis de travailler efficacement et d'avoir un suivi clair sur les différentes tâches que chacun effectuait. A la fin de chaque sprint, nous avions la réunion rétrospective avec nos référents, afin de faire un bilan sur le sprint et les deux semaines de travail et de pouvoir recueillir leurs remarques et conseils. Nous avons choisi de tous travailler ensemble sur l'ensemble du projet et de ne pas nous répartir les tâches selon le domaine d'activité pour que chacun puisse développer et apprendre sur différents aspects de la réalisation d'une application mobile et acquérir diverses compétences qui pourront tous nous être utiles.

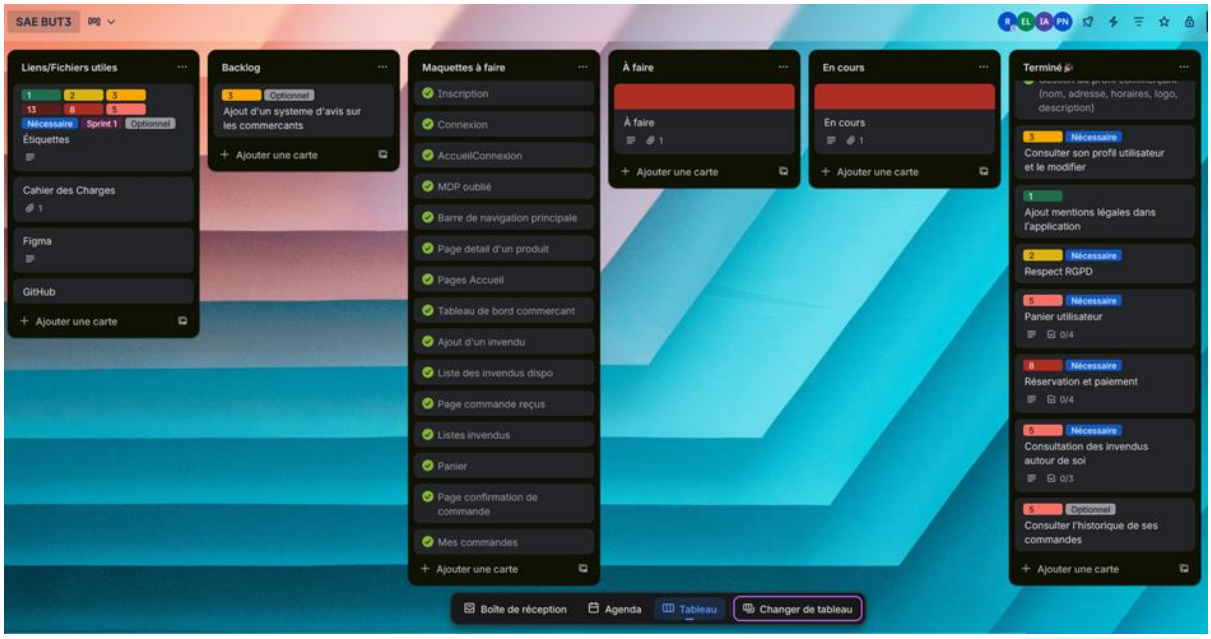


Figure 1 : Tableau Trello de gestion de projet

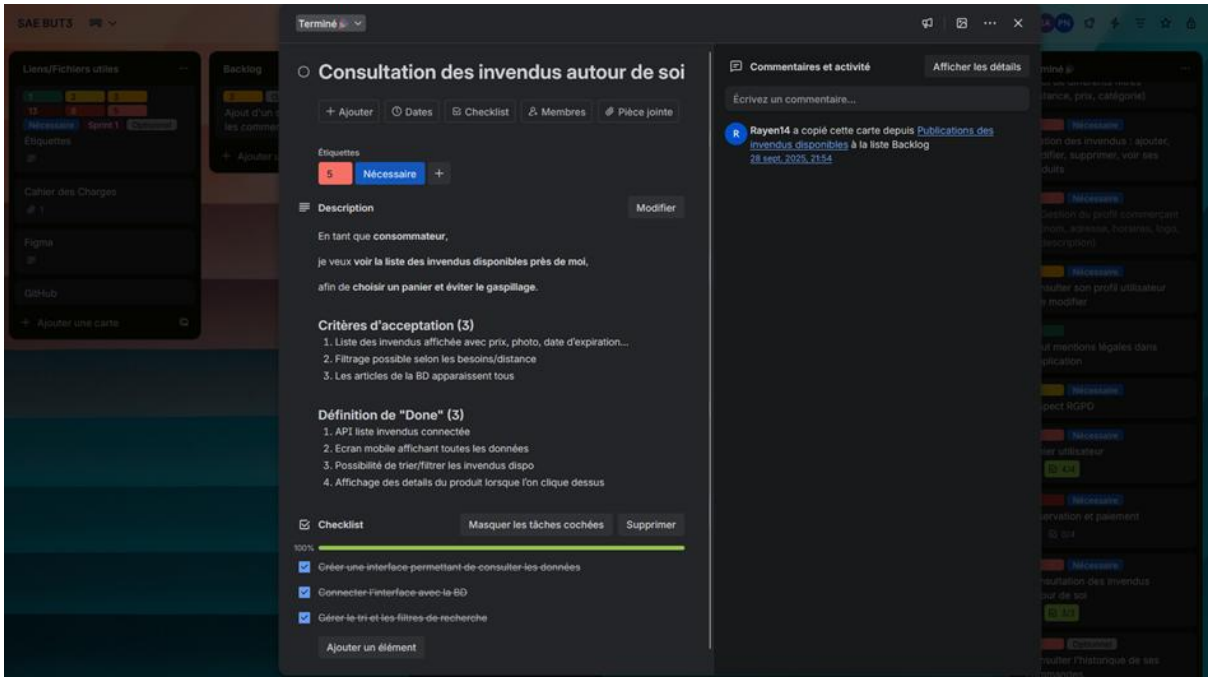


Figure 2 : Tableau Trello de gestion de projet, avec détail des scénarios d'utilisateur, critères d'acceptation et definition of done d'une tâche

3.2 Outils de collaboration et technique

Pour mener à bien ce projet et suivre l'organisation, nous avons utilisé plusieurs outils de collaboration :

- **Git & GitHub :** Nous avons utilisé Git pour gérer les différentes versions de notre code mais aussi pour fusionner nos codes. Cela a été crucial pour travailler simultanément sur le Frontend et le Backend, mais aussi pour nous permettre de travailler ensemble simultanément sur des tâches différentes facilement.
- **Trello :** Nous avons utilisé un tableau Trello afin de suivre l'avancée du projet et de suivre la méthodologie Scrum plus facilement grâce au système de listes et de tickets.
- **Discord :** Nous avons utilisé la plateforme Discord pour la communication du projet, que ce soit par écrit mais également pour des réunions que nous avons faites pour nous mettre d'accord et avancer ensemble sur le développement. Elle nous a aussi servi d'espace d'échanges et de stockage de fichiers.
- **Google Docs :** Cet outil nous a permis de travailler ensemble afin de rédiger et de garder les différentes idées, le cahier des charges mais aussi les synthèses de réunion, et les rapports.
- **Figma** : Figma nous a permis de faire les maquettes avant de commencer le développement, afin d'avoir un aperçu des différentes pages de l'application.

3.3 Déroulement des sprints

Le développement a été découpé en plusieurs sprints de deux semaines. Les premiers sprints ont été consacrés à la mise en place des fondations (structure du projet, navigation, authentification, base de données et API). Les sprints suivants ont permis d'implémenter les fonctionnalités principales (gestion des produits côté vendeur, recherche/filtrage, panier et commande, paiement Stripe) puis les fonctionnalités d'accompagnement (recettes, recherche par ingrédients et propositions d'articles proches). Enfin, une phase de stabilisation a été consacrée aux tests, corrections et améliorations de l'expérience utilisateur.

4. Cahier des charges et besoins

Cette section synthétise les besoins identifiés au lancement du projet ECO STOCK ainsi que les contraintes à respecter. Elle s'appuie sur le cahier des charges rédigé en début de SAÉ, qui définit le périmètre, les fonctionnalités attendues et les critères de validation.

4.1 Besoins fonctionnels

4.1.1 Acteurs et cas d'usage principaux

Le cahier des charges identifie deux types d'utilisateurs principaux : les commerçants et les consommateurs. Les commerçants publient des invendus (photo, quantité, prix, date de péremption, description, stock et la catégorie) et suivent leurs ventes, tandis que les consommateurs consultent les offres, réservent ou achètent et accèdent à un onglet "recettes". Dans l'évolution du projet, un troisième acteur a été ajouté : les associations, disposant de comptes dédiés leur permettant d'accéder à des offres spécifiques ou réservées afin de faciliter la redistribution des invendus.

4.1.2 Fonctionnalités indispensables

Les fonctionnalités jugées indispensables au bon fonctionnement de l'application sont :

- Création de compte et authentification pour accéder aux fonctionnalités.
- Gestion du profil (informations, adresse, photo).
- Recherche et consultation des offres (produits disponibles autour de l'utilisateur, commerces proches).
- Ajout au panier, réservation et achat de produits.
- Publication des invendus par les commerçants (avec détails de l'offre).
- Paiement : soit en magasin, soit en ligne via Stripe.
- Recommandations personnalisées de produits (basées sur les préférences, localisation, achats).
- Onglet "recettes" : suggestions adaptées aux produits sélectionnés afin d'encourager l'achat et limiter le gaspillage.
- Espace vendeur : publier et gérer ses invendus (création, modification, suppression/désactivation, stock).
- Comptes associations : accès à des offres spécifiques/réservées pour faciliter la redistribution des invendus.
- Historique d'achats et de commandes.
- Recommandations personnalisées (préférences, localisation, historique).
- Filtres par régimes alimentaires (végétarien, sans gluten…).
- Recherche intelligente (par ingrédients/mots-clés) avec tolérance aux variations d'écriture grâce un calcul de différence entre deux mots.

4.1.3 Fonctionnalités optionnelles

Certaines fonctionnalités étaient envisagées comme des améliorations possibles :

- Système de notes/avis.
- Notifications (commandes, nouveautés, rappels).
- Espace communautaire pour partager des recettes.

4.2 Contraintes techniques et non fonctionnelles

4.2.1 Contraintes techniques

Le cahier des charges fixe une architecture basée sur :

- une application mobile **React Native** (multi-plateforme) avec **TypeScript/JSX** pour l'interface.
- un back-end **Node.js/Express** exposant une **API REST** en **JSON**.
- une base de données **PostgreSQL** pour stocker utilisateurs, commerce, produits, paniers, commande, recettes, ingrédients, la table qui lie les recettes avec ingrédients et la table qui lit des produits avec des ingrédients.
- une authentification basée sur **JWT** pour sécuriser les sessions.
- l'intégration de **Stripe** pour le paiement en ligne (avec possibilité de tests).
- Architecture client/serveur : **appli mobile → API REST → base de données.**
- Stockage sécurisé du token côté mobile (**SecureStore**).
- Gestion des images (stockée via **Cloudinary**).
- mise en place d'un module de matching textuel (similarité de chaînes) pour la recherche par ingrédients.
- Géolocalisation / cartographie

React Native a choisi retenu afin de développer une application mobile multi-plateforme à partir d'une base de code unique, permettant ainsi de réduire le temps de développement et de maintenance. Node.js, associé au framework Express, a permis la mise en place d'une API REST légère, performante et adaptée aux besoins d'un projet en temps réel.

PostgreSQL est utilisé pour sa robustesse, sa fiabilité et son respect de l'intégrité transactionnelle, ainsi que pour sa capacité à gérer des relations complexes entre les différentes entités de l'application (produits, commandes, ingrédients). Stripe a été intégré afin de garantir des paiements en ligne sécurisés, tandis que Cloudinary a été utilisé pour assurer une gestion optimisée des images, sans alourdir la base de données.

4.2.2 Contraintes non fonctionnelles

Plusieurs exigences non fonctionnelles ont été identifiées :

- performance (application fluide même en présence de nombreuses offres) et approche mobile.
- respect du RGPD pour la protection des données personnelles.
- attention à l'accessibilité (lisibilité, contrastes, navigation simple).
- une démarche de durabilité : limiter les requêtes inutiles et encourager des pratiques responsables via l'application.

- Sécurité (hash du mot de passe, routes protégées, contrôle des rôles)
- Fiabilité (gestion des erreurs, validation des entrées)

4.3 Critères de validation

4.3.1 Indicateurs de succès

Le projet est considéré comme validé si :

- Un utilisateur peut s'inscrire/se connecter et accéder aux fonctionnalités sans blocage.
- un consommateur peut consulter et rechercher des offres (catégories / ingrédients / proximité), afficher le détail (prix, stock, DLC) et ajouter au panier et à la possibilité de filtrer selon plusieurs critères (gluten, distance, dlc, prix).
- La commande peut être validée avec un paiement en ligne (Stripe) aboutissant à la création de la commande en base et à la mise à jour du stock.
- A partir d'un panier proposer de recettes liés aux ingrédients à l'intérieurs
- Un commerçant peut publier et gérer ses invendus (création, modification/désactivation) et consulter ses ventes/commandes.
- L'onglet Recettes affiche des suggestions exploitables, c'est-à-dire que lorsqu'on clique sur le détail d'une recette, il y a la possibilité de cliquer sur les ingrédients disponibles et que ça nous propose celui le plus proche de nous.
- Une association peut accéder aux offres réservées.
- l'application est jugée stable, simple et compréhensible lors de tests utilisateurs (pas de bug bloquant).
- la recherche par ingrédients retourne des résultats pertinents même en cas de variations d'écriture (ex : accents, pluriels, fautes simples).
- L'utilisateur peut consulter "Mes achats" (historique) et marquer une commande comme récupérée.
- La fiche produit affiche la localisation du commerce (carte) et les informations de retrait.
- Une offre peut être réservée aux associations et un compte association peut y accéder.
- L'utilisateur peut gérer un moyen de paiement.
- À l'inscription, l'utilisateur peut choisir un type de compte (Client / Vendeur / Association).
- L'utilisateur peut modifier ses informations de profil (informations personnelles, adresse).
- Un vendeur peut créer une offre complète incluant catégorie, ingrédients, lot/panier surprise, horaires et instructions de retrait.
- Un vendeur peut accéder à ses produits et consulter ses ventes/commande

---

4.3.2 Tests et procédure de validation

La validation du projet s'appuie sur une stratégie de tests automatisés mise en place afin de s'assurer du bon fonctionnement et de la fiabilité de l'application. Les tests ont été réalisés à l'aide du framework **Jest**, avec **ts-jest** pour l'exécution des tests écrits en TypeScript.

Cette démarche de test concerne l'ensemble de l'application, aussi bien le **back-end** que le **front-end**, ainsi que les interactions entre les différents modules. Elle permet de vérifier les règles métier, les calculs, la gestion des données, la sécurité et la communication avec l'API, avant même toute interaction avec l'interface utilisateur.

Côté back-end, les tests portent sur les principales fonctionnalités de l'API, telles que la gestion des utilisateurs et des commandes, les calculs métier, la pagination, la géolocalisation (notamment via la formule de Haversine) ainsi que les mécanismes de sécurisation reposant sur les tokens JWT.
Côté front-end, les tests valident la logique métier indépendamment de l'interface React Native : validations de formulaires, gestion du panier, calculs de prix et de réductions, filtrage des offres, affichage de conseils écologiques et appels aux services API.

Les tests de l'interface utilisateur sont réalisé manuellement.

L'ensemble des suites de tests a été exécuté avec succès avec un taux de réussite de **100 %**, ce qui confirme la stabilité globale de l'application. Le détail des tests unitaires front-end et back-end, ainsi que des tests d'intégration, est présenté dans la partie 6 – Tests et validation, où sont décrits les scénarios testés, les outils utilisés et les résultats obtenus.

5. Conception et solutions mises en œuvre

5.1 Architecture générale (mobile ↔ API ↔ base de données)

ECO STOCK repose sur une architecture client/serveur classique :

- Application mobile (React Native / TypeScript) : interface utilisateur, navigation, stockage sécurisé de session, accès géolocalisé aux offres, panier, paiement, recettes.

- API REST (Node.js / Express) : centralise la logique métier (authentification, produits, commandes, ingrédients, recettes), sécurise les accès via JWT, contrôle des droits selon le type de compte.

- Base de données PostgreSQL : persistance des utilisateurs, commerces, produits, ingrédients, recettes, commandes, etc.

Les échanges se font en JSON via des endpoints organisés par domaine (/auth, /products, /seller, /recipes, /payment, etc.). Le serveur expose aussi un endpoint “racine” listant les routes disponibles et un healthcheck pour vérifier l’état du service.

```
Eco_Front/
├── src/
│   ├── screens/           # Écrans de l'application
│   │   ├── auth/          # Connexion, Inscription, ResetPassword
│   │   ├── HomeScreen.tsx # Page d'accueil + liste produits
│   │   ├── CartScreen.tsx # Panier avec validation stock
│   │   ├── ProfileScreen.tsx
│   │   ├── RecipesScreen.tsx
│   │   ├── PaymentScreen.tsx
│   │   ├── SettingsScreen.tsx
│   │   ├── AddProductScreen.tsx
│   │   └── *.styles.ts    # Fichiers de styles associés
│   ├── navigation/        # Navigation React Navigation
│   │   ├── RootNavigator.tsx
│   │   ├── HomeNavigator.tsx
│   │   ├── ProfileNavigator.tsx
│   │   └── types.ts       # Types TypeScript navigation
│   ├── components/        # Composants réutilisables
│   │   ├── ProductCard.tsx
│   │   ├── RecipeCard.tsx
│   │   ├── LoadingSpinner.tsx
│   │   └── ...
│   ├── contexts/          # React Contexts (state global)
│   │   ├── AuthContext.tsx
│   │   ├── CartContext.tsx
│   │   └── ThemeContext.tsx
│   ├── services/          # Logique métier & API
│   │   └── api.ts         # Client API REST (250+ lignes)
│   ├── utils/             # Utilitaires & helpers
│   │   ├── ecoTips.ts     # 20 conseils écologiques
│   │   ├── validations.ts # Validation formulaires
│   │   └── formatters.ts  # Formatage dates, prix
│   ├── constants/         # Constantes globales
│   │   ├── categories.ts  # Catégories produits
│   │   ├── colors.ts      # Palette de couleurs
│   │   └── config.ts      # Configuration app
│   └── theme/             # Thème & styles globaux
│       ├── colors.ts      # Couleurs principales
│       ├── spacing.ts     # Espacements
│       └── typography.ts  # Polices & tailles
├── __tests__/             # Suite de tests (167 tests)
│   ├── utils/
│   │   ├── validations.test.ts
│   │   ├── cart-helpers.test.ts
│   │   ├── auth-helpers.test.ts
│   │   ├── recipes-helpers.test.ts
│   │   ├── geolocation-helpers.test.ts
│   │   └── ecoTips.test.ts
│   └── services/
│       ├── api.test.ts
│       ├── api-functions.test.ts
│       └── pagination.test.ts
├── assets/                # Images, fonts, icons
├── jest.config.js
├── jest.setup.simple.js
└── package.json
```

Diagramme d'architecture simplifié Front

```
ecostock_backend/
├── routes/                 # Routes API
│   ├── auth.js             # POST /register, /login, /change-password
│   ├── products.js         # CRUD produits + recherche paginée
│   ├── payment.js          # Stripe payment intents
│   ├── recipes.js          # Suggestions recettes anti-gaspi
│   ├── ingredients.js      # Gestion ingrédients
│   ├── profile.js          # Profil utilisateur (upload photo)
│   └── seller.js           # Routes vendeur (produits, commandes)
├── middleware/             # Middleware Express
│   ├── auth.js             # Vérification JWT (authenticateToken)
│   └── checkUserType.js    # Autorisation par rôle (vendeur, acheteur)
├── services/               # Services externes
│   └── cloudinary.js       # Upload images Cloudinary
├── utils/                  # Utilitaires
│   ├── cleanupExpiredProducts.js  # Cleanup produits expirés
│   └── fuzzyMatch.js       # Recherche floue (fuzzy search)
├── config/
│   └── database.js         # Connexion PostgreSQL (pg pool)
├── __tests__/              # Suite de tests (83 tests)
│   ├── routes/
│   │   └── auth.test.js    # Tests routes authentification (25)
│   ├── middleware/
│   │   └── auth.test.js    # Tests middleware JWT (17)
│   └── utils/
│       └── business-logic.test.js  # Tests logique métier (41)
├── .env                    # Variables d'environnement (non versionné)
├── server.js               # Point d'entrée (Express app)
├── jest.config.js
└── package.json
```

Diagramme d'architecture simplifié Back

5.1.1 Organisation et structure du code

L'organisation du code source a été pensée pour garantir la maintenabilité, la lisibilité et la scalabilité du projet. Une séparation claire des responsabilités a été appliquée à la fois au niveau de l'architecture globale (frontend/backend) et au sein de chaque partie du projet.

5.1.2 Séparation frontend / backend

Le projet est organisé en deux dépôts distincts, chacun avec ses propres dépendances, configurations et logiques :

Frontend (Eco_Front/) : contient exclusivement l'application mobile React Native, avec ses composants visuels, sa navigation, son state management et ses styles. Ce découplage permet au frontend de rester indépendant du backend, facilitant les tests UI isolés et

permettant potentiellement de remplacer ou faire évoluer l'API sans impacter l'interface mobile.

Backend (ecostock_backend/) : contient l'API REST Node.js/Express, la logique métier, les middlewares de sécurité, et les scripts utilitaires. Cette séparation garantit que la logique sensible (authentification, calculs, accès base de données) reste côté serveur et ne peut pas être manipulée depuis le client.

5.1.3 Structure modulaire du frontend

L'application mobile suit une organisation par domaines fonctionnels, respectant le principe de séparation des responsabilités (Single of responsability):

src/components/ : contient tous les composants réutilisables (ProductCard, FilterModal, Button, Input, EcoTip, etc.). Chaque composant est autonome et peut être importé dans plusieurs écrans différents. Les composants complexes disposent de leur propre fichier de styles (ex: FilterModal.styles.ts) pour séparer la logique métier du style.

src/screens/ : regroupe tous les écrans de l'application, organisés par fonctionnalité (auth/, produits, panier, profil, commandes, recettes). Chaque écran possède deux fichiers : un fichier principal (.tsx) pour la logique et le rendu, et un fichier de styles (.styles.ts) pour toute la mise en forme. Cette séparation rend le code plus lisible et permet de modifier l'apparence sans toucher à la logique métier.

src/navigation/ : centralise toute la configuration de React Navigation avec des navigateurs séparés par domaine (AuthNavigator pour l'authentification, MainTabNavigator pour les onglets principaux, ProfileNavigator pour le profil, etc.). Le fichier types.ts définit les types TypeScript pour les paramètres de navigation, garantissant la sécurité de typage lors des navigations entre écrans.

src/contexts/ : contient les contextes React (Context API) pour le state management global. Le CartContext gère l'état du panier de manière centralisée, permettant à tous les écrans d'accéder et de modifier le panier directement sans ce compliqué le travail.

src/services/ : regroupe tous les appels API dans un fichier centralisé (api.ts). Toutes les requêtes HTTP vers le backend passent par ce service unique, facilitant la maintenance (si l'URL de l'API change, un seul fichier est à modifier) et permettant d'ajouter facilement des gestion des erreurs, ajout automatique du token d'authentification, logs.

src/theme/ : définit le design system de l'application avec des fichiers séparés pour les couleurs (colors.ts), les espacements (spacing.ts), et la typographie (typography.ts). Cette centralisation garantit la cohérence visuelle de toute l'application et facilite les modifications de design (changement de palette de couleurs en un seul endroit).

src/utils/ : contient les fonctions utilitaires réutilisables (ecoTips.ts pour les messages écologiques aléatoires, fonctions de formatage de dates, calculs, etc.).

5.1.4 Structure du backend par responsabilités

Le backend suit le pattern architectural MVC (Model-View-Controller) adapté au contexte API REST, avec une organisation claire par couches :

routes/ : contient la définition de tous les endpoints REST, organisés par domaine fonctionnel. Chaque fichier de route (auth.js, products.js, seller.js, recipes.js, payment.js, etc.) regroupe les endpoints liés à une fonctionnalité spécifique. Les routes définissent uniquement les chemins HTTP et délèguent la logique métier aux middlewares et aux contrôleurs. Cette organisation facilite la lecture de l'API : en consultant le fichier routes/products.js, on comprend immédiatement tous les endpoints disponibles pour gérer les produits.

middleware/ : regroupe tous les middlewares Express qui s'exécutent avant les contrôleurs de routes. Le fichier auth.js contient le middleware authenticateToken qui vérifie la validité du JWT sur chaque requête protégée. Le fichier checkUserType.js vérifie le type d'utilisateur (client/vendeur/association) pour restreindre l'accès à certaines routes selon les rôles. Cette centralisation des contrôles de sécurité évite la duplication de code et garantit une application uniforme des règles de sécurité sur toute l'API.

config/ : contient les fichiers de configuration centralisés, notamment database.js qui gère la connexion à PostgreSQL via un pool de connexions. Cette séparation permet de changer facilement les paramètres de connexion (base de données de développement vs production) sans toucher au code métier.

utils/ : regroupe les fonctions utilitaires réutilisables côté serveur, comme cleanupExpiredProducts.js (script de nettoyage automatique des produits expirés) et fuzzyMatch.js (algorithme de recherche floue pour les ingrédients). Ces utilitaires peuvent être appelés depuis plusieurs routes ou exécutés de manière indépendante (cron jobs).

services/ (prévu) : bien que partiellement implémenté, ce dossier est destiné à contenir les services métier complexes comme cloudinary.js pour la gestion des uploads d'images. Cette couche de services permet d'abstraire la logique métier complexe hors des routes, les rendant plus légères et faciles à tester.

Le point d'entrée unique **server.js** configure Express avec tous les middlewares globaux (CORS, parseur JSON, logger), enregistre toutes les routes, et démarre le serveur. Cette centralisation facilite la compréhension du flux de démarrage de l'application.

5.1.5 Séparation stricte styles / logique (frontend)

Une convention forte a été appliquée sur le frontend : **aucun style inline dans les composants**. Tous les styles sont extraits dans des fichiers séparés portant l'extension .styles.ts, utilisant l'API StyleSheet de React Native.

Par exemple, pour l'écran SearchScreen :

- **SearchScreen.tsx** : contient uniquement la logique (state, hooks, fonctions, rendu JSX)
- **SearchScreen.styles.ts** : contient tous les styles dans un objet StyleSheet

Cette séparation offre plusieurs avantages :

- **Lisibilité** : le fichier principal reste concentré sur la logique, sans être pollué par des centaines de lignes de styles
- **Réutilisabilité** : les styles peuvent être partagés entre plusieurs composants si nécessaire
- **Performance** : StyleSheet.create() optimise les styles en les créant une seule fois au lieu de les recréer à chaque rendu
- **Maintenance** : un designer peut modifier l'apparence en ne touchant qu'aux fichiers .styles.ts sans risquer de casser la logique

---

5.1.6 Typage strict avec TypeScript

L'utilisation de TypeScript sur le frontend garantit la sécurité de typage et réduit drastiquement les erreurs à l'exécution. Tous les composants, fonctions, et objets sont typés explicitement :

- **Types de navigation** (types.ts) : définissent les paramètres attendus pour chaque écran, empêchant les erreurs de navigation avec de mauvais paramètres
- **Interfaces de données** : définissent la structure exacte des objets retournés par l'API (Product, User, Recipe, Order, etc.)
- **Props de composants** : chaque composant réutilisable définit une interface pour ses props, documentant son utilisation et empêchant les erreurs de passage de props

---

5.1.7 Gestion des variables d'environnement et secrets

Toutes les informations sensibles sont externalisées dans des fichiers d'environnement :

Backend (.env) : contient JWT_SECRET, les clés Stripe (STRIPE_SECRET_KEY, STRIPE_PUBLISHABLE_KEY), les credentials Cloudinary (CLOUD_NAME, API_KEY, API_SECRET), et les paramètres de connexion PostgreSQL (DB_USER, DB_PASSWORD, DB_HOST, DB_NAME, DB_PORT). Ce fichier est exclu du contrôle de version via .gitignore.

Frontend (environnement Expo) : contient l'URL de l'API backend, les clés publiques Stripe, et autres configurations. Ces variables sont injectées au moment du build via EAS (Expo Application Services).

Un fichier d'exemple (.env.example) est fourni avec des valeurs factices pour guider la configuration lors du clonage du projet. Cette approche garantit que les secrets ne sont jamais commités dans le dépôt Git.

5.1.8 Commentaires et documentation du code

Le code est organisé de manière à être auto-documenté (noms de variables explicites, fonctions courtes avec une responsabilité unique), des commentaires ont été ajoutés aux endroits nécessaires.

5.2 Conception de la base de données (description textuelle des liens)

La base a été structurée pour séparer clairement les responsabilités : comptes, catalogue, ingrédients/recettes, transactions et aspects "sociaux" (associations).

Utilisateurs, rôles, associations et commerces :

La table **users** centralise tous les comptes utilisateurs du système avec un champ discriminant user_type qui peut prendre trois valeurs : 'client', 'vendeur' ou 'association'. Chaque utilisateur possède un identifiant unique de type UUID généré automatiquement, une adresse email unique servant d'identifiant de connexion, et un mot de passe haché avec bcrypt (10 salt rounds). Les informations de profil (prénom, nom, téléphone, adresse complète avec ville et code postal) sont optionnelles pour respecter le principe de minimisation des données RGPD. Un champ is_active (booléen) permet une désactivation logique du compte (soft delete) sans suppression physique, préservant ainsi l'intégrité des données historiques. Les champs created_at et updated_at sont gérés automatiquement par des triggers PostgreSQL pour tracer l'historique.

Pour les vendeurs, une table séparée **commerces** stocke les informations spécifiques de leur point de vente : nom du commerce, adresse physique complète, coordonnées géographiques précises (latitude en DECIMAL(10,8) et longitude en DECIMAL(11,8) pour une précision métrique), numéro de téléphone, description textuelle, horaires d'ouverture en format texte libre, et un flag is_active. Cette table est liée à la table users par une clé étrangère vendeur_id référençant l'ID de l'utilisateur vendeur. Une contrainte d'unicité (UNIQUE) sur vendeur_id garantit qu'un vendeur ne peut gérer qu'un seul commerce actif. La suppression d'un compte vendeur entraîne automatiquement la suppression de son commerce associé grâce à la clause ON DELETE CASCADE sur la clé étrangère.

Produits, catégories et ingrédients :

La table **produits** constitue le cœur du catalogue de l'application. Chaque produit possède un identifiant unique UUID, un nom et une description détaillée obligatoire (pour préciser les

quantités exactes, le conditionnement et les détails du lot). Deux champs de prix sont présents : prix qui représente le prix réduit proposé sur la plateforme (DECIMAL(10,2)), et prix_original qui stocke le prix normal avant réduction, permettant de calculer et d'afficher le pourcentage d'économie réalisée.

Le champ dlc (DATE) stocke la date limite de consommation, critère majeur pour le tri et le filtrage des produits proches de leur péremption. Chaque produit est rattaché à une catégorie via category_id (clé étrangère vers la table categories) et à un commerce via commerce_id (clé étrangère vers commerces avec ON DELETE CASCADE). Le stock disponible est géré par un champ INTEGER stock qui représente la quantité d'unités disponibles, décrémenté automatiquement lors des achats. Un flag booléen is_disponible permet de retirer temporairement un produit de la vente sans le supprimer physiquement.
Les horaires de retrait peuvent être précisés via deux champs optionnels pickup_start_time et pickup_end_time (type TIME) pour indiquer aux clients les plages horaires pendant lesquelles ils peuvent récupérer leurs produits. L'URL de l'image du produit est stockée dans le champ image_url (TEXT), cette image étant hébergée sur Cloudinary et non dans la base de données.

La table **categories** contient une liste fixe prédéfinie de 8 catégories : 'Fruits et légumes', 'Produits laitiers', 'Viande et poisson', 'Pain et viennoiseries', 'Épicerie', 'Surgelés', 'Boissons', et 'Autre'. Cette normalisation facilite le filtrage rapide et l'organisation claire de l'offre. Chaque catégorie possède un ID auto-incrémenté (SERIAL), un nom unique, et optionnellement une description.

La table **ingredients** stocke les noms d'ingrédients avec une éventuelle catégorisation. Les noms sont normalisés (minuscules, sans accents) pour faciliter les recherches. Une table de jonction **product_ingredients** crée la relation many-to-many entre produits et ingrédients, permettant de taguer chaque produit avec ses ingrédients constitutifs. Cette table contient les clés étrangères product_id et ingredient_id, avec une contrainte UNIQUE sur le couple (product_id, ingredient_id) pour éviter les doublons. La suppression d'un produit entraîne automatiquement la suppression de ses associations d'ingrédients grâce au ON DELETE CASCADE.

Recettes et leurs ingrédients :

La table **recettes** stocke les recettes proposées aux utilisateurs avec : un identifiant auto-incrémenté (SERIAL PRIMARY KEY), un nom, une description courte présentant le plat, des instructions détaillées de préparation (TEXT), un temps de préparation en minutes (INTEGER), un niveau de difficulté contrôlé par une contrainte CHECK acceptant uniquement 'facile', 'moyen' ou 'difficile', et une URL d'image hébergée sur Cloudinary.
La table de jonction **recette_ingredients** associe chaque recette à la liste de ses ingrédients requis. Elle contient les clés étrangères recette_id et ingredient_id, ainsi qu'une colonne quantite en format texte libre (VARCHAR) permettant de spécifier des quantités comme "200g", "2 pièces", "1 cuillère à soupe", etc. Une contrainte UNIQUE(recette_id, ingredient_id) empêche les doublons. Les deux clés étrangères sont configurées avec ON DELETE CASCADE pour maintenir l'intégrité référentielle : la suppression d'une recette supprime automatiquement toutes ses associations d'ingrédients.

L'algorithme de matching de recettes fonctionne par intersection : il compte combien d'ingrédients d'une recette sont présents dans le panier ou les produits consultés par l'utilisateur, puis calcule un score de correspondance en pourcentage (nombre d'ingrédients présents / nombre total d'ingrédients de la recette × 100). Les recettes sont ensuite triées par score décroissant pour proposer en priorité celles que l'utilisateur peut réaliser avec le plus d'ingrédients déjà disponibles.

Commandes et système de paiement :

La table **commandes** gère le cycle de vie complet d'une commande. Chaque commande possède un identifiant UUID unique, un numéro de commande généré automatiquement selon le format CMD-YYYY-NNNNNN par un trigger PostgreSQL utilisant une séquence dédiée commandes_sequence (exemple : CMD-2026-000042). La fonction PL/pgSQL generate_numero_commande() s'exécute avant chaque insertion pour construire ce numéro en concaténant 'CMD-', l'année courante formatée sur 4 chiffres, et le prochain numéro de séquence padé sur 6 chiffres avec des zéros à gauche.
Chaque commande référence un client via client_id (clé étrangère vers users) et un commerce via commerce_id (clé étrangère vers commerces), tous deux avec ON DELETE CASCADE. Le montant total de la commande est pré-calculé et stocké dans le champ total (DECIMAL(10,2)).

Le statut de la commande évolue selon un workflow défini, contrôlé par une contrainte CHECK acceptant uniquement : 'en_attente' (lors de la création après paiement), 'confirmee' (après validation par le vendeur), 'prete' (quand le produit est disponible au retrait), 'recuperee' (quand le client a effectivement récupéré sa commande), ou 'annulee' (en cas d'annulation).

L'intégration avec Stripe est matérialisée par trois champs : stripe_payment_intent_id (VARCHAR(255)) stocke l'identifiant unique du Payment Intent Stripe, stripe_payment_status (VARCHAR(50)) reflète le statut côté Stripe (requires_payment_method, requires_confirmation, requires_action, processing, succeeded, canceled), et paid_at (TIMESTAMP) horodate précisément le moment où le paiement a été effectué avec succès.

La table **commande_items** détaille le contenu de chaque commande en stockant les articles achetés. Elle contient une clé étrangère commande_id (avec ON DELETE CASCADE) vers la table commandes, une clé étrangère product_id vers la table produits, la quantite achetée (INTEGER), et le prix_unitaire (DECIMAL(10,2)) figé au moment de l'achat. Ce dernier point est crucial : en stockant le prix au moment de la transaction, on évite les incohérences si le vendeur modifie ultérieurement le prix du produit. Ainsi, l'historique des commandes reflète toujours le prix réellement payé par le client.

Panier :

Bien que le panier soit principalement géré en local sur l'application mobile via le CartContext React (state management avec Context API), la conception de la base de données prévoit la possibilité d'une persistance serveur future pour permettre la synchronisation multi-device. Actuellement, le panier est stocké dans SecureStore côté mobile avec la structure suivante : un tableau d'objets contenant pour chaque article le product_id, la quantité sélectionnée, et toutes les informations du produit pré-chargées (nom, prix, image, commerce, etc.) pour éviter

des appels API répétés. À chaque modification du panier (ajout, suppression, changement de quantité), une sauvegarde dans SecureStore est déclenchée, permettant la persistance même si l'application est fermée. Au prochain lancement, le panier est automatiquement restauré depuis le stockage local.

Optimisations : index et vues SQL :

Pour améliorer significativement les performances des requêtes fréquentes, plusieurs index B-tree ont été créés sur les colonnes utilisées dans les clauses WHERE, JOIN et ORDER BY :

- **idx_produits_category** sur produits(category_id) : accélère les filtres par catégorie
- **idx_produits_commerce** sur produits(commerce_id) : optimise la récupération des produits d'un commerce spécifique
- **idx_produits_dlc** sur produits(dlc) : améliore le tri par date de péremption et le filtrage des produits proches de leur DLC
- **idx_produits_disponible** sur produits(is_disponible) : accélère l'exclusion des produits indisponibles
- **idx_commandes_client** sur commandes(client_id) : optimise la récupération de l'historique d'achat d'un client
- **idx_commandes_commerce** sur commandes(commerce_id) : accélère l'affichage des commandes reçues par un vendeur
- **idx_commandes_statut** sur commandes(statut) : facilite le filtrage des commandes par état

Une vue SQL matérialisée **user_orders_with_pickup** effectue les jointures complexes entre les tables commandes, commande_items, produits, commerces et users. Cette vue permet de récupérer en une seule requête optimisée toutes les informations nécessaires à l'affichage d'une commande : les détails de chaque produit commandé (nom, image, DLC), les informations du commerce (nom, adresse, coordonnées GPS), les horaires de retrait, les montants par ligne et le total général. Cette approche réduit drastiquement le nombre de requêtes SQL nécessaires côté application et améliore les temps de réponse de l'API.

5.3 Sécurité : authentification, sessions, contrôles d’accès, protection des secrets

La sécurité a été traitée à plusieurs niveaux.

5.3.1 Protection des secrets (fichier .env)

Les éléments sensibles (clés Stripe, secret JWT, identifiants Cloudinary, etc.) ne sont pas codés en dur.
Ils sont chargés via un fichier .env (exposé sous forme d’exemple .env.example) et récupérés côté serveur avec dotenv.
Objectif : éviter qu’un dépôt ou un build contient des secrets.

5.3.2 Authentification JWT + expiration de session

Après login/register, l'API délivre un JWT signé (JWT_SECRET) avec une durée configurée à 90 jours par défaut (expiresIn: '90d').
Côté mobile, le token est stocké via Expo SecureStore (stockage chiffré / sécurisé).
À chaque appel API, le token est envoyé dans l'entête : Authorization: Bearer <token>.

Si le token est expiré ou invalide, l'application :

- efface les données sécurisées (token + user).
- affiche une alerte "session expirée".
- redirige vers l'authentification.

Résultat : après environ 90 jours, l'utilisateur est automatiquement déconnecté.

5.3.3 Hash des mots de passe

Les mots de passe ne sont jamais stockés en clair :

- hash via bcrypt (salt rounds 10)
- vérification via bcrypt.compare au login.

5.3.4 Middlewares de contrôle d'accès

L'API sécurise ses routes avec des middlewares :

- authenticateToken : vérifie le JWT, récupère l'utilisateur en base, refuse si inactif.
- contrôle du type (vendeur, client, association) avant certaines actions :
  - routes vendeur pour publier/modifier/supprimer.
  - règles d'affichage des offres "réservées aux associations".

Ces contrôles empêchent qu'un utilisateur non autorisé accède à des actions critiques.

5.3.5 Validation et cohérence des entrées

L'API applique des règles métier strictes :

- champs obligatoires lors de la publication de produit.
- description obligatoire (afin d'indiquer les quantités/détails).
- vérification du stock et de la disponibilité au moment du paiement.
- prévention des incohérences (ex : stock insuffisant).

5.4 Gestion des images avec Cloudinary (éviter le base64 en BDD)

Au lieu de stocker des images en base (base64 lourd, coûteux et lent à récupérer), ECO STOCK utilise Cloudinary.
La gestion des images au sein de l'application repose sur l'utilisation du service Cloudinary. Lorsqu'une image est ajoutée depuis l'application mobile, celle-ci est envoyée au serveur sous forme de données encodées (généralement au format base64). Le back-end se charge

ensuite de transmettre cette image à Cloudinary, qui l'héberge et retourne une URL sécurisée.

Seule cette URL est enregistrée en base de données, notamment dans le champ products.image_url pour les produits ou dans les champs correspondants pour les images de profil des utilisateurs. L'image elle-même n'est donc jamais stockée directement dans la base de données.

Ce choix présente plusieurs avantages : il permet de réduire la taille de la base de données, d'améliorer les temps de chargement grâce à la diffusion optimisée des images et de bénéficier automatiquement des fonctionnalités de Cloudinary telles que l'optimisation de la qualité et du format des images. De plus, la maintenance est simplifiée puisque les images peuvent être supprimées ou mises à jour facilement en cas de modification ou de suppression d'un produit.

5.5 Géolocalisation et calcul des distances/temps d'accès

5.5.1 Position utilisateur → offres proches

L'application récupère la position (latitude/longitude) et la transmet à l'API lors des appels produits.
L'API calcule une distance (km) avec une formule de type Haversine.

5.5.2 Temps estimés : à pied, vélo et transport

Pour améliorer l'expérience, l'API renvoie aussi des estimations :

- à pied (environ 5 km/h),
- à vélo (environ 15 km/h),
- en transport (environ 20 km/h) mais pour les distances très courtes (< 2km), marcher est considéré plus rapide.

L'objectif est de proposer une information utile même sans dépendre directement d'une API externe de navigation. De plus elle propose des solutions écologiques au debut (à pieds, vélo) puis le trajet le plus court;

5.5.3 Géolocalisation

La fiche détaillée d'un produit intègre une visualisation cartographique du commerce, permettant à l'utilisateur de localiser précisément le point de retrait. Cette information est accompagnée des horaires de récupération définis par le vendeur.

5.6 Logique "DLC" + fenêtre de retrait (anti-gaspillage réaliste)

5.6.1 Règles de publication vendeur

Lorsqu'un vendeur publie un produit :

- la DLC doit être au minimum demain (pas aujourd'hui / pas passée),
- une description est obligatoire (quantités, détails : "lot de 3 pommes", "1kg de tomates"…),
- Le vendeur définit une heure de début et une heure de fin de retrait si cette heure est dépassé alors le produit devient inactif

---

5.6.2 Désactivation automatique des ventes "hors délai"

Un produit n'est plus considéré disponible si :

- la DLC est dépassée,
- ou si l'heure de fin de retrait (pickup_end_time) est dépassée.

Cela se traduit directement dans les requêtes de listing/recherche (produits filtrés si pickup_end_time < current_time). Donc l'application retire automatiquement de l'offre les produits trop tard à récupérer.

5.7 Recherche intelligente et tolérante aux fautes (ingrédients + description)

Pour faciliter la recherche, ECO STOCK combine plusieurs approches :

5.7.1 Recherche multi-champs

La recherche produits (et recettes) prend en compte :

- nom du produit
- description
- catégorie
- ingrédients liés au produit

5.7.2 "Fuzzy matching" (correction approximative)

Afin d'améliorer l'autocomplétion des ingrédients, notamment côté vendeur lors de l'ajout ou de la modification d'un produit, un mécanisme de recherche approximative (fuzzy matching) a été mis en place. Ce mécanisme vise à proposer des correspondances pertinentes même lorsque la saisie de l'utilisateur n'est pas exacte.

Le traitement s'effectue en plusieurs étapes. Dans un premier temps, les chaînes de caractères sont normalisées (conversion en minuscules, suppression des accents et nettoyage des caractères non pertinents). Une recherche est ensuite réalisée selon différents niveaux de correspondance : correspondance exacte, correspondance par début de mot, puis correspondance par inclusion. Si aucune correspondance satisfaisante n'est trouvée, un calcul de similarité est effectué à l'aide de la distance de Levenshtein, permettant de mesurer la proximité entre deux mots.

Cette approche permet de proposer des ingrédients pertinents même en cas de fautes de frappe (par exemple “tomatte” au lieu de “tomate”), d’absence d’accents (“pate” au lieu de “pâte”) ou de variations de forme telles que le pluriel.

Ainsi, le système de recherche devient plus tolérant, inclusif et robuste, améliorant significativement l’expérience utilisateur et réduisant les erreurs lors de la saisie des ingrédients.

5.8 Lots (panier surprise) vs produit simple : logique et impact recettes

5.8.1 Gestion des offres

ECO STOCK gère deux formes d’offres afin d’adapter la gestion des invendus et la logique de recommandations/recettes. Une offre peut correspondre à un produit unique ou à un lot (panier) regroupant plusieurs ingrédients.

Produit simple (is_lot = false) :

- associé à un ingrédient principal (via product_items) ;
- facilite des suggestions ciblées (ex. “il manque des œufs pour la recette”).

Lot / panier (is_lot = true) :

- associé à plusieurs ingrédients (plusieurs lignes dans product_items) ;
- cohérent avec le concept de panier tout en conservant la transparence (composition précisée via la description et les ingrédients associés).

5.8.2 Gestion des commandes côté client

L’application propose un écran « Mes achats » permettant au consommateur de consulter l’ensemble de ses commandes passées. Les commandes sont organisées selon leur statut (non récupérées / récupérées), facilitant le suivi des retraits.

Une fois le produit récupéré en magasin, l’utilisateur peut marquer la commande comme récupérée. Cette action déclenche une mise à jour du statut côté serveur, garantissant la cohérence entre l’interface utilisateur et les données stockées en base.

5.8.3 Gestion des commandes côté vendeur

Côté vendeur, un écran dédié permet de consulter les commandes reçues et de suivre l’état des ventes. Cette fonctionnalité offre une visibilité claire sur les produits vendus et les commandes à honorer, facilitant l’organisation du retrait et la gestion des invendus.

5.9 Recettes : lien entre ce que l’utilisateur achète et comment l’utiliser

La fonctionnalité “Recettes” est pensée comme une continuité logique après l’achat.

5.9.1 Trouver des recettes à partir d’ingrédients

L’API permet de proposer différentes méthodes d’accès aux recettes afin de s’adapter aux besoins et aux usages des utilisateurs. Elle peut notamment retourner des recettes aléatoires, paginées afin de faciliter la navigation et la découverte. Une recherche par titre ou par ingrédients est également disponible, permettant à l’utilisateur de cibler précisément une recette.

Enfin, un mode de recherche spécifique par ingrédients a été mis en place. Dans ce cas, les recettes sont classées selon un score de correspondance, calculé à partir du nombre d’ingrédients communs entre la recette et ceux sélectionnés ou disponibles. Cette approche favorise les recettes les plus pertinentes et valorise les produits déjà présents dans le panier de l’utilisateur.

5.9.2 Suggestion d’achats complémentaires depuis une recette (proximité)

Depuis le détail d’une recette, l’utilisateur peut interagir directement avec la liste des ingrédients. Lorsqu’un ingrédient est sélectionné, l’application appelle un endpoint dédié chargé de rechercher les produits correspondants proposés par les commerçants.

Les résultats sont ensuite triés par proximité géographique lorsque la localisation de l’utilisateur est disponible, afin de proposer en priorité les produits les plus proches. Cette logique permet de faciliter la complétion du panier tout en favorisant les commerces locaux.

Cette fonctionnalité illustre la démarche globale du projet, qui vise à accompagner l’utilisateur de l’achat jusqu’à la consommation, en lui proposant des solutions pertinentes et contextualisées pour compléter ses recettes sans générer de déplacements inutiles.

5.10 Paiement en ligne et finalisation de commande

Le paiement des commandes est géré via l’intégration du service Stripe, utilisé de manière simplifiée afin de permettre le règlement en ligne des produits réservés. Lorsqu’un utilisateur valide son panier, l’application déclenche la création d’un PaymentIntent côté serveur.

Une fois le paiement confirmé, une confirmation visuelle est affichée à l’utilisateur. La commande est alors enregistrée en base de données, le stock des produits concernés est mis à jour et la commande apparaît dans l’historique des achats du client. Cette étape marque la finalisation du parcours d’achat au sein de l’application.

5.11 Associations : offres réservées et accès différencié

Afin de renforcer l'impact social de l'application, ECO STOCK intègre un mécanisme d'accès différencié aux offres pour les associations. Certains produits peuvent être marqués comme réservés aux associations grâce à un indicateur dédié (reserved_for_associations).

Les utilisateurs disposant d'un compte de type association peuvent consulter et accéder à ces offres spécifiques, tandis que les utilisateurs de type client ne les voient pas apparaître dans la liste standard des produits disponibles. Cette distinction permet de faciliter la redistribution des invendus vers des structures engagées dans l'aide alimentaire, tout en évitant toute confusion pour les autres utilisateurs.

Cette règle est appliquée au niveau de l'API, via un filtrage côté serveur. Ce choix garantit une meilleure sécurité et empêche tout contournement côté client, assurant ainsi une gestion fiable et cohérente des accès aux offres réservées.

5.12 Frontend : organisation, navigation, séparation UI / logique

5.12.1 Navigation

L'application mobile s'appuie sur une navigation structurée, organisée autour de différents espaces fonctionnels tels que l'authentification, l'accueil, la recherche ou le profil utilisateur. Cette organisation permet de clarifier les parcours, depuis la connexion jusqu'à l'utilisation des fonctionnalités principales. Elle contribue également à isoler les écrans dédiés aux vendeurs, notamment pour l'ajout de produits, la gestion des commandes et des invendus. Enfin, cette structuration rend l'application évolutive, en facilitant l'ajout de nouvelles fonctionnalités sans impacter les parcours existants.

L'interface repose sur une navigation par onglets et écrans dédiés, assurant une séparation claire entre les fonctionnalités client, vendeur et association, et facilitant la compréhension des parcours utilisateurs.

5.12.2 Séparation des styles

Les écrans et composants séparent les styles dans des fichiers *.styles.ts.
Objectif : code plus lisible, réutilisable, et facile à maintenir (UI séparée de la logique métier).

5.12.3 Panier et logique centralisée

La logique panier est factorisée via un CartContext (contexte global), ce qui permet :

- d'éviter de dupliquer des calculs,

- de partager le panier entre écrans (liste → détail → paiement).

5.12.4 Messages de sensibilisation (identité écologique)

Des messages “eco tips” apparaissent à certains moments (ex : chargement / connexion). Ce n’est pas qu’un élément UI : c’est un choix de conception pour renforcer l’identité du projet et maintenir l’utilisateur dans une démarche de réduction du gaspillage.

5.13 Pagination et optimisation des performances

Pour gérer efficacement l'affichage de longues listes de produits sans dégrader les performances, ECO STOCK implémente un système de pagination avec scroll infini côté frontend.

5.13.1 Principe du scroll infini

Au chargement initial, seul un premier lot de produits (20 à 30) est récupéré et affiché. Lorsque l'utilisateur scrolle et approche de la fin de la liste, un événement déclenche automatiquement le chargement du lot suivant. Le composant FlatList de React Native expose les props onEndReached et onEndReachedThreshold (configuré à 0.5 pour anticiper le chargement).

5.13.2 Gestion côté API

L'API backend supporte ce mécanisme via les query parameters page et limit. Le backend calcule l'offset avec la formule (page - 1) * limit et utilise les clauses SQL LIMIT et OFFSET pour paginer les résultats. La réponse JSON inclut des métadonnées : total (nombre total de produits), page (page actuelle), pages (nombre total de pages), et hasMore (booléen indiquant s'il reste des pages).

5.13.3 État et synchronisation client

Un état local tracking le numéro de page courante et un flag loading empêchent les chargements multiples simultanés. Lorsque onEndReached est déclenché, l'application vérifie que loading est false et hasMore est true avant d'incrémenter le compteur de page. Les nouveaux produits sont concaténés à la liste existante pour créer l'illusion d'un scroll continu. Un ActivityIndicator s'affiche en bas de la liste pendant la récupération. En cas d'erreur réseau, un message permet de réessayer. Ce pattern est appliqué sur les listes de produits, recettes, et commandes.

5.14 Gestion avancée du stock et prévention des conflits

5.14.1 Problématique de la concurrence

La gestion distribuée du stock nécessite des mécanismes robustes pour gérer les situations où plusieurs utilisateurs tentent d'acheter simultanément le dernier exemplaire d'un produit, ou lorsqu'un utilisateur garde un produit en panier pendant des heures alors que le stock a été écoulé entre-temps.

5.14.2 Vérification atomique au paiement

Le stock n'est jamais décrémenté au moment de l'ajout au panier, mais seulement au paiement confirmé. Lors de la création du Payment Intent, une transaction PostgreSQL commence avec BEGIN, sélectionne les produits avec un verrou exclusif FOR UPDATE, vérifie que stock >= quantité demandée et is_disponible = true. En cas de succès, le Payment Intent est créé ; sinon, la transaction est rollback avec un message d'erreur détaillé (exemple : 'Le produit Pain aux céréales n'a plus que 1 unité disponible').

5.14.3 Finalisation et idempotence

Une fois le paiement confirmé par Stripe, la même transaction reprend, décrémente le stock (UPDATE produits SET stock = stock - quantité), met à jour is_disponible = false si le nouveau stock vaut 0, crée l'enregistrement de commande et ses items, puis commit. L'atomicité garantit que toutes ces opérations réussissent ou aucune. Un mécanisme d'idempotence basé sur stripe_payment_intent_id empêche la création de commandes en double en cas de timeout réseau.

5.15 Calcul et affichage des réductions

Pour inciter les utilisateurs à l'achat et valoriser l'économie réalisée, l'application calcule et affiche automatiquement le pourcentage de réduction par rapport au prix original. Chaque produit possède deux champs : prix (prix réduit proposé, ex: 2.50€) et prix_original (prix normal avant réduction, ex: 5.00€).
Le pourcentage de réduction est calculé côté frontend avec la formule : ((prix_original - prix) / prix_original) * 100. Ce pourcentage est arrondi à l'entier le plus proche et affiché dans un badge coloré visible (ex: '-50%') pour attirer l'attention. Au niveau du panier, le total des économies réalisées est calculé en sommant (prix_original - prix) * quantité pour chaque article, avec un message valorisant comme 'Vous économisez 12.50€ avec cette commande !'. Cette gamification de l'économie renforce le sentiment de faire une bonne affaire tout en participant à une démarche écoresponsable.

5.16 Synchronisation et persistance du panier

Le panier d'achat est géré globalement dans l'application mobile via le CartContext, un Context API React qui maintient l'état du panier et expose des fonctions pour ajouter, retirer ou modifier des articles.
L'état contient un tableau cartItems avec pour chaque article : product_id, quantité, et toutes les informations du produit pré-chargées (nom, prix, image, etc.) pour éviter des appels API répétés. Le contexte expose les fonctions addToCart(product, quantity), removeFromCart(productId), updateQuantity(productId, newQuantity), et clearCart().
Chaque modification déclenche une sauvegarde asynchrone dans AsyncStorage, permettant la persistance même si l'application est fermée. Au prochain lancement, le panier est restauré depuis AsyncStorage (usage de asyncStorage car les données ne sont pas sensibles). Un refresh périodique ou manuel vérifie la disponibilité et les prix actuels. Si un produit n'est plus disponible ou son prix a changé, un badge d'avertissement s'affiche avec possibilité de retirer

l'article ou d'accepter le nouveau prix. Le CartContext est wrappé dans App.tsx, rendant le panier accessible depuis n'importe quel écran via le hook useCart, évitant le props drilling.

5.17 Index et optimisations de la base de données

Pour améliorer les performances des requêtes fréquentes, plusieurs index ont été créés sur les colonnes utilisées dans les clauses WHERE, JOIN et ORDER BY :
• idx_produits_category : sur produits(category_id) pour les filtres par catégorie
• idx_produits_commerce : sur produits(commerce_id) pour récupérer rapidement les produits d'un commerce
• idx_produits_dlc : sur produits(dlc) pour trier par date de péremption
• idx_produits_disponible : sur produits(is_disponible) pour exclure les produits indisponibles
• idx_commandes_client : sur commandes(client_id) pour l'historique client
• idx_commandes_commerce : sur commandes(commerce_id) pour les commandes vendeur
• idx_commandes_statut : sur commandes(statut) pour filtrer par état
Une vue SQL user_orders_with_pickup effectue les jointures complexes entre commandes, commande_items, produits, commerces et users, permettant de récupérer en une seule requête toutes les informations nécessaires à l'affichage d'une commande (détails produits, informations commerce, horaires de retrait, montants).

5.18 Triggers et automatisations PostgreSQL

Plusieurs triggers PostgreSQL automatisent certaines tâches répétitives et garantissent la cohérence des données :

5.18.1 Mise à jour automatique de updated_at

Une fonction PL/pgSQL update_updated_at_column() met automatiquement à jour le champ updated_at avec CURRENT_TIMESTAMP à chaque modification d'enregistrement. Cette fonction est associée à des triggers BEFORE UPDATE sur les tables users, produits et commandes, permettant de tracer automatiquement l'historique des modifications sans intervention manuelle dans le code applicatif.

5.18.2 Génération automatique des numéros de commande

Une séquence dédiée commandes_sequence et une fonction generate_numero_commande() génèrent automatiquement un numéro de commande unique au format CMD-YYYY-NNNNNN (exemple : CMD-2026-000042) à chaque insertion dans la table commandes. Le trigger set_numero_commande s'exécute BEFORE INSERT et construit le numéro en concaténant le préfixe 'CMD-', l'année courante, et le prochain numéro de séquence padé sur 6 chiffres. Ce mécanisme garantit l'unicité, la lisibilité et la traçabilité chronologique des commandes.

6. Tests et validation

Dans le cadre de notre démarche Agile Scrum, nous avons adopté une approche de développement guidée par les tests (Test-Driven Development). À la fin de chaque sprint, nous avons systématiquement amélioré et enrichi nos tests pour valider les nouvelles fonctionnalités (User Stories) avant de les intégrer. Cette méthode nous a permis de détecter rapidement les régressions et de garantir la stabilité de l'application ECO STOCK tout au long de son développement.

6.1 Tests continus

Afin de valider le bon fonctionnement des composants, nous avons mis en place une architecture de tests automatisés utilisant le framework Jest. Ces tests étaient exécutés systématiquement avant chaque commit important et lors de l'intégration de nouvelles fonctionnalités dans la branche develop.

L'application ECO STOCK dispose d'une suite de tests exhaustive avec 250 tests unitaires répartis entre le frontend et le backend, garantissant la fiabilité et la qualité du code sur l'ensemble du stack technique.

Statistiques globales :

- Total : 250 tests (100% de réussite)
- Frontend : 167 tests - Logique métier React Native/TypeScript
- Backend : 83 tests - API Node.js/Express et logique serveur
- Temps d'exécution total : environ 8 secondes

Cette couverture exhaustive permet de détecter les régressions, valider les règles métier et garantir la sécurité des données utilisateurs.

Notre couverture de tests s'étend sur l'ensemble des fonctionnalités critiques de l'application :

- Gestion des utilisateurs : Vérification de la sécurité des données d'inscription et de connexion
- Logique métier du e-commerce : Calculs des prix, gestion des stocks, application des réductions et calcul des totaux
- Gestion des dates : Algorithmes de vérification des Dates Limites de Consommation (DLC) pour identifier les produits périmés ou urgents
- Moteur de recherche : Nettoyage des requêtes utilisateurs et construction dynamique des filtres URL
- Fonctionnalités écologiques : Logique d'affichage aléatoire des conseils anti-gaspillage (ecoTips)
- Géolocalisation : Calculs de distance avec la formule de Haversine et tri par proximité
- Système de recettes : Algorithme de matching d'ingrédients et scoring de correspondance
- Pagination : Gestion du scroll infini et calculs des métadonnées

6.1.1 Tests Unitaires Frontend (167 tests)

Les tests unitaires frontend couvrent la logique métier de l'application mobile React Native, indépendamment de l'UI, afin de garantir la fiabilité des calculs, validations et règles business.

Validations de formulaires (32 tests)

- Validation des emails (RFC 5322), téléphones français (10 chiffres) et mots de passe (≥ 6 caractères, caractères spéciaux pour mots de passe forts).
- Validation des prix (positifs, format décimal, prix réduit ≤ prix initial).
- Validation des stocks (entiers positifs, pas de valeurs négatives).
- Validation des dates DLC (format ISO, date future).

Authentification et sécurité (24 tests)

- Vérification du format des JWT (3 parties) et détection de l'expiration via le champ `exp`.
- Redirection vers la connexion en cas de session expirée.
- Validation de la force et de la confirmation des mots de passe.
- Validation des types d'utilisateurs (acheteur, vendeur, association) pour l'accès aux fonctionnalités.

Logique du panier (26 tests)

- Calculs financiers : sous-totaux, total commande, pourcentage de réduction et économies réalisées.
- Validation de la disponibilité (stock > 0, `is_disponible = true`).
- Prévention du dépassement de stock et validation des quantités (min 1, max stock).
- Garantie de l'absence d'erreurs d'arrondi.

Système de recettes (22 tests)

- Filtrage par recherche textuelle, catégorie et ingrédients disponibles.
- Calcul du score de correspondance (% d'ingrédients disponibles).
- Détection des ingrédients manquants.
- Validation des données de recette (nom, ingrédients, étapes).

- Calcul et formatage du temps de préparation (ex. 90 min → 1h30).

Géolocalisation (20 tests)

- Validation de la formule de Haversine avec coordonnées réelles (marge < 1%).
- Tri des commerces par distance et filtrage par rayon (5, 10, 20 km).
- Validation des coordonnées GPS (lat/lon).
- Formatage des distances (mètres ou km avec 2 décimales).

Pagination et infinite scroll (25 tests)

- Calcul du nombre total de pages et des offsets SQL.
- Navigation entre pages (suivant, précédent, saut direct).
- Gestion des états première/dernière page.
- Infinite scroll : déclenchement correct, prévention des appels multiples et des doublons.
- Gestion des états vides.

API et services (22 tests)

- Construction dynamique des URLs avec paramètres de filtres.
- Formatage des données (dates ISO, prix décimaux, distances).
- Génération aléatoire d'eco-tips sans répétition immédiate.
- Gestion et formatage des erreurs réseau.

6.1.2 Tests Unitaires Backend (83 tests)

Les tests backend valident l'API Node.js/Express, la sécurité, la validation des données et les règles métier serveur.

Routes d'authentification (41 tests)

POST /api/auth/register (13 tests)

- Inscription réussie pour acheteur, vendeur (avec infos commerce) et association.

- Rejets : email invalide/manquant, mot de passe trop court, email déjà utilisé, vendeur sans commerce.
- Transaction SQL atomique (BEGIN/COMMIT).
- Génération automatique d'un JWT après inscription.

POST /api/auth/login (14 tests)

- Connexion réussie avec retour des infos spécifiques (commerce ou association).
- Rejets : champs manquants, email inexistant, mot de passe incorrect, compte inactif.
- Génération d'un JWT avec expiration à 90 jours.

POST /api/auth/change-password (14 tests)

- Changement de mot de passe avec JWT valide.
- Rejets : token manquant/invalide/expiré, mot de passe actuel incorrect, utilisateur inexistant.
- Validation du nouveau mot de passe et hachage bcrypt avant stockage.

Middleware d'authentification (29 tests)
Validation du token (12 tests)

- Rejet sans Authorization ou format invalide.
- Rejet des tokens invalides, expirés ou liés à un utilisateur inexistant/inactif.
- Injection sécurisée de `req.user` sans exposer le mot de passe.

Sécurité JWT (10 tests)

- Validation de la signature avec `JWT_SECRET`.
- Détection des tokens falsifiés ou modifiés.
- Validation du payload et gestion des erreurs de décodage.

Base de données (7 tests)

- Requêtes filtrées par `is_active = true`.
- Gestion des erreurs SQL (500).
- Support des trois types d'utilisateurs.

Logique métier backend (13 tests)
Validation des prix (6 tests)

- Prix strictement positifs.
- Prix réduit ≤ prix initial.
- Calcul correct des réductions et économies.
- Rejet des incohérences et valeurs invalides.

Gestion du stock (6 tests)

- Stock ≥ 0, détection des ruptures et stocks bas.
- Calcul du stock restant après commande.
- Prévention de l'overselling.

Disponibilité produit (1 test)

- Produit disponible uniquement si stock > 0, DLC future et `is_disponible = true`.

6.1.3 Tests d'Intégration (Services et Contextes)

Ces tests valident l'interaction entre modules et la gestion de l'état global.

Contexte du panier (CartContext)

- Simulation complète du parcours d'achat : ajout, mise à jour des quantités (selon stock), suppression.
- Persistance du panier via mock d'AsyncStorage.
- Vérification de la synchronisation des composants consommateurs via la Context API.

Services API

- Validation de la construction des URLs avec filtres (query params).
- Injection automatique du JWT dans les en-têtes Authorization.
- Gestion des erreurs réseau (timeouts, messages utilisateur clairs).
- Utilisation de mocks API sans dépendance réseau ou base PostgreSQL active.

6.1.4 Méthodologie de Test

Technologies utilisées :

- Jest : Framework de test JavaScript/TypeScript universel
- ts-jest : Préprocesseur TypeScript pour Jest permettant de tester du code TypeScript
- Supertest : Bibliothèque de tests HTTP pour Express permettant de simuler des requêtes
- Mocks : Simulation de la base de données PostgreSQL et d'AsyncStorage pour tester sans dépendances externes

Principes appliqués :

- Tests unitaires purs : Aucun appel réseau ou base de données réel pendant les tests
- Isolation : Chaque test est complètement indépendant
- Rapidité : Exécution complète en quelques secondes pour un feedback immédiat

Bénéfices de cette couverture :

- Détection de régressions : Tout changement cassant la logique métier est immédiatement détecté lors du lancement des tests
- Refactoring sécurisé : Possibilité de remanier le code pour améliorer sa structure sans crainte de casser des fonctionnalités
- Qualité garantie : 250 vérifications automatiques avant chaque commit important
- Sécurité renforcée : Validation JWT, bcrypt, protection contre les injections SQL
- Conformité métier : Toutes les règles de gestion sont validées automatiquement

Bilan global des tests : Grâce à cette approche, il y a 250 tests au total (167 frontend + 83 backend), tous validés avec succès à 100%. L'exécution complète des tests prend environ 8 secondes, ce qui nous a permis de les lancer à chaque modification du code pour détecter instantanément la moindre régression.

6.2 Validation des Performances

Une fois l'application fonctionnelle, nous avons procédé à des tests de performance pour valider sa capacité à monter en charge et à offrir une expérience fluide aux utilisateurs.

Tests de charge :

Simulation de 50 utilisateurs simultanés consultant des produits, avec temps de réponse moyen maintenu sous 200 millisecondes pour les requêtes de listing.

Tests sur les endpoints critiques (création de Payment Intent Stripe, finalisation de commande avec décrémentation de stock) avec vérification de l'absence de perte de données et du respect de l'atomicité des transactions. Validation que le système de verrous FOR UPDATE empêche bien les conditions de course même sous charge élevée.

```
PASS  __tests__/utils/auth-helpers.test.ts
PASS  __tests__/utils/recipes-helpers.test.ts
PASS  __tests__/utils/geolocation-helpers.test.ts
PASS  __tests__/utils/cart-helpers.test.ts
PASS  __tests__/utils/ecoTips.test.ts
PASS  __tests__/services/api-functions.test.ts
PASS  __tests__/utils/validations.test.ts
PASS  __tests__/services/pagination.test.ts
PASS  __tests__/services/api.test.ts

Test Suites: 9 passed, 9 total
Tests:       167 passed, 167 total
Snapshots:   0 total
Time:        5.101 s
Ran all test suites.

C:\Users\limon\Appmobile\Eco_Stock\Eco_Front>cd ..

C:\Users\limon\Appmobile\Eco_Stock>cd ecostock_backend

C:\Users\limon\Appmobile\Eco_Stock\ecostock_backend>npm test

> ecostock-backend@1.0.0 test
> jest

PASS  __tests__/utils/business-logic.test.js
PASS  __tests__/middleware/auth.test.js
PASS  __tests__/routes/auth.test.js

Test Suites: 3 passed, 3 total
Tests:       83 passed, 83 total
Snapshots:   0 total
Time:        2.59 s
Ran all test suites.

C:\Users\limon\Appmobile\Eco_Stock\ecostock_backend>
```

6.2.1 Backend : Résistance à la charge

Nous avons soumis notre API Node.js à un test de stress ("Stress Test") utilisant l'outil Apache Bench (ab). Le scénario consistait à simuler une forte affluence sur une route complexe (recherche de recettes par ingrédients) nécessitant des calculs et des accès en base de données. Nous avons testé 1000 requêtes envoyées avec une concurrence de 50 utilisateurs simultanés, la requête testée est la recherche de recettes avec comme liste d'ingrédients farine, œuf et lait.

Résultats obtenus :

- **Stabilité :** Le serveur a affiché un taux de réussite de **100%**
- **Débit :** L'API est capable de traiter **912 requêtes par seconde**
- **Latence :** Le temps de réponse moyen est de **54 ms**, avec 99% des requêtes servies en moins de 88 ms.

Ces chiffres démontrent que l'architecture backend est robuste et dimensionnée pour supporter un trafic utilisateur conséquent.

```
rayen@rayen-MS-7C95:~$ ab -n 1000 -c 50 -H "Authorization: Bearer $TOKEN" "http://localhost:3000/api/recipes/by-ingredients?ingredients=farine,oeuf,lait"
This is ApacheBench, Version 2.3 <$Revision: 1903618 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient)
Completed 100 requests
Completed 200 requests
Completed 300 requests
Completed 400 requests
Completed 500 requests
Completed 600 requests
Completed 700 requests
Completed 800 requests
Completed 900 requests
Completed 1000 requests
Finished 1000 requests


Server Software:
Server Hostname:        localhost
Server Port:            3000

Document Path:          /api/recipes/by-ingredients?ingredients=farine,oeuf,lait
Document Length:        2915 bytes

Concurrency Level:      50
Time taken for tests:   1.096 seconds
Complete requests:      1000
Failed requests:        0
Total transferred:      3157000 bytes
HTML transferred:       2915000 bytes
Requests per second:    912.17 [#/sec] (mean)
Time per request:       54.814 [ms] (mean)
Time per request:       1.096 [ms] (mean, across all concurrent requests)
Transfer rate:          2812.22 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.5      0       3
Processing:    15   53   7.7     51      98
Waiting:       12   52   7.6     51      98
Total:         15   53   7.9     51      99

Percentage of the requests served within a certain time (ms)
  50%     51
  66%     53
  75%     56
  80%     58
  90%     63
  95%     66
  98%     74
  99%     88
 100%     99 (longest request)
```

6.2.2 Base de Données : Optimisation des requêtes

La rapidité du backend repose en grande partie sur l'optimisation de la base de données PostgreSQL. Nous avons analysé le plan d'exécution des requêtes SQL critiques (notamment le filtrage des produits par prix) à l'aide de la commande EXPLAIN ANALYZE.

Comme le montre l'analyse ci-dessous, le temps d'exécution réel ("Execution Time") est extrêmement faible, de l'ordre de 0.033 ms. Bien que le moteur utilise un "Seq Scan" sur notre jeu de données de test (ce qui est le comportement optimal pour de petits volumes), la structure est prête à utiliser les Index créés (idx_products_prix) dès que le volume de données augmentera.

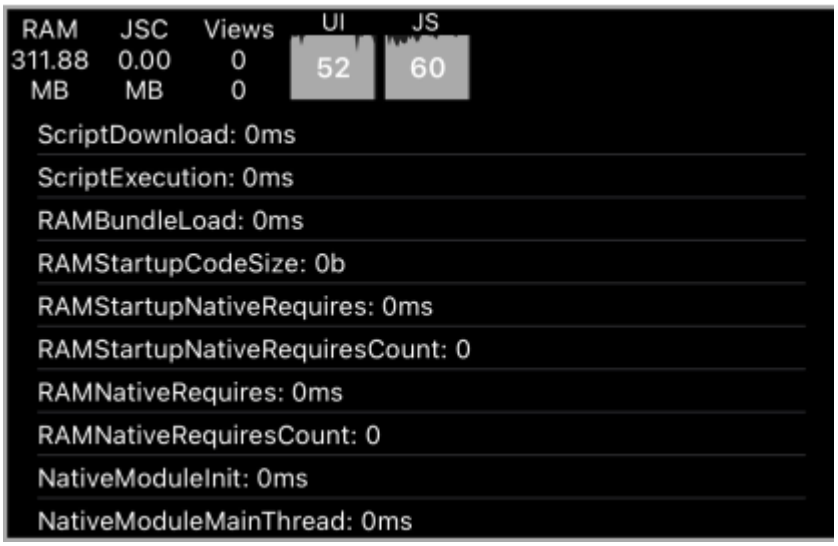
```
rayen@rayen-MS-7C95:~$ sudo -u postgres psql -d ecostock -c "EXPLAIN ANALYZE SELECT nom, prix FROM products WHERE prix < 10;"
[sudo] Mot de passe de rayen :
                                          QUERY PLAN
-----------------------------------------------------------------------------------------------
 Seq Scan on products  (cost=0.00..1.16 rows=4 width=532) (actual time=0.014..0.017 rows=13 loops=1)
   Filter: (prix < '10'::numeric)
 Planning Time: 0.352 ms
 Execution Time: 0.033 ms
(4 rows)
```

6.2.3 Frontend : Fluidité et Expérience Utilisateur

Enfin, nous avons validé la performance de l'interface mobile React Native à l'aide du **Performance Monitor** intégré à Expo.

- **Fluidité (FPS) :** Lors du défilement des listes de produits (qui sont les éléments les plus gourmands en ressources), l'application maintient un taux de rafraîchissement constant de **60 FPS** (UI et JS threads).

- **Mémoire (RAM) :** La consommation mémoire reste stable, sans fuite détectée lors de la navigation entre les écrans.



7. Résultats obtenus et analyse

Le projet ECO STOCK a abouti à la réalisation d'une application mobile fonctionnelle répondant aux objectifs fixés dans le cahier des charges. Cette section présente les résultats obtenus tant sur le plan fonctionnel que technique, ainsi qu'une analyse des performances et de l'adéquation du projet aux besoins initialement identifiés.

7.1 Fonctionnalités implémentées et validation des objectifs

L'ensemble des fonctionnalités définies comme indispensables dans le cahier des charges a été implémenté avec succès.

7.1.1 Gestion des comptes et authentification

Le système d'authentification repose sur des tokens JWT avec une durée de validité de 90 jours, stockés de manière sécurisée via SecureStore. L'application distingue trois types d'utilisateurs (client, vendeur, association) avec des interfaces et des autorisations adaptées à chaque rôle. La création de compte vendeur intègre la saisie des informations du commerce, incluant l'autocomplétion d'adresse via l'API Nominatim pour garantir des coordonnées GPS précises.

7.1.2 Publication et gestion des offres par les vendeurs

Les vendeurs disposent d'une interface complète leur permettant de publier des produits avec l'ensemble des informations nécessaires : photo (hébergée sur Cloudinary), description détaillée, prix original et prix réduit, date limite de consommation, stock disponible, catégorie et ingrédients. Le système impose des contraintes réalistes, notamment une DLC minimale fixée au lendemain et l'obligation de préciser des horaires de retrait. Les vendeurs peuvent modifier ou désactiver leurs offres, et consulter leurs commandes en temps réel avec un affichage clair du statut de chaque commande.

7.1.3 Consultation et recherche de produits pour les consommateurs

L'écran d'accueil affiche les produits disponibles avec un système de pagination et de scroll infini permettant une navigation fluide même avec plusieurs centaines de produits en base. Les utilisateurs bénéficient de plusieurs modes de recherche et de filtrage : recherche textuelle avec tolérance aux fautes grâce à l'algorithme de distance de Levenshtein, filtres par catégorie, par ingrédients, par proximité géographique (avec calcul de distance via la formule de Haversine), par date limite de consommation, et par prix. Le tri par proximité propose des estimations de temps de trajet à pied, à vélo et en transport en commun, favorisant les déplacements écologiques.

7.1.4 Système de panier et processus d'achat

La gestion du panier est centralisée via un Context React permettant une synchronisation en temps réel entre tous les écrans de l'application. Le panier est persisté localement via AsyncStorage, garantissant sa conservation même après fermeture de l'application. Le système affiche clairement le montant total, le montant des économies réalisées et le pourcentage de réduction pour chaque produit. Le paiement en ligne est intégré via Stripe avec création d'un Payment Intent sécurisé. Des vérifications atomiques côté serveur, protégées par des verrous PostgreSQL (FOR UPDATE), empêchent les problèmes de concurrence sur le stock et garantissent qu'un produit ne peut être vendu qu'une seule fois.

7.1.5 Fonctionnalité recettes et suggestions personnalisées

L'onglet recettes propose un catalogue de recettes avec recherche par titre ou par ingrédients. Un algorithme de scoring calcule automatiquement la correspondance entre les ingrédients du panier de l'utilisateur et ceux requis pour chaque recette, affichant un pourcentage de compatibilité. Depuis le détail d'une recette, l'utilisateur peut cliquer directement sur un ingrédient manquant pour découvrir les produits correspondants disponibles à proximité, facilitant ainsi la complétion des achats. Cette fonctionnalité accompagne véritablement l'utilisateur de l'achat jusqu'à la consommation et valorise les produits achetés en proposant des idées concrètes d'utilisation.

7.1.6 Dimension sociale : comptes associations et offres réservées

Le système de comptes associations a été pleinement implémenté avec un filtrage côté serveur garantissant que seuls les utilisateurs disposant du type "association" peuvent accéder aux offres marquées comme réservées. Cette fonctionnalité renforce la dimension sociale du projet en facilitant la redistribution des invendus vers les structures d'aide alimentaire, tout en maintenant une séparation claire avec l'offre grand public.

7.1.7 Historique des commandes et suivi

Les utilisateurs disposent d'un écran "Mes achats" listant l'ensemble de leurs commandes passées avec un affichage détaillé des produits, des montants, des statuts et des informations de retrait. Une fois le produit récupéré en magasin, le client peut marquer la commande comme récupérée, déclenchant une mise à jour du statut côté serveur. Côté vendeur, un écran dédié permet de consulter les commandes reçues et d'en suivre l'état, facilitant l'organisation du retrait.

7.2 Performances techniques et optimisations

Les tests de performance réalisés démontrent que l'application répond aux exigences de fluidité et de réactivité attendues d'une application mobile moderne.

7.2.1 Performance du backend

Les tests de charge avec Apache Bench (1000 requêtes, 50 utilisateurs concurrents) sur l'endpoint de recherche de recettes par ingrédients ont donné les résultats suivants :

- **Taux de réussite** : 100% (aucune erreur HTTP)
- **Débit** : 912 requêtes par seconde
- **Latence moyenne** : 54 ms
- **99e percentile** : moins de 88 ms

Ces résultats démontrent que l'architecture backend est robuste et capable de supporter un trafic utilisateur important sans dégradation significative des performances.

7.2.2 Optimisation de la base de données

L'analyse du plan d'exécution des requêtes SQL via EXPLAIN ANALYZE révèle des temps d'exécution très faibles, de l'ordre de 0,033 ms pour une requête de filtrage de produits par prix. Les index créés sur les colonnes fréquemment utilisées dans les clauses WHERE, JOIN et ORDER BY (category_id, commerce_id, dlc, is_disponible, client_id, statut) garantissent des performances optimales même avec un volume de données croissant. La vue SQL user_orders_with_pickup, qui effectue les jointures complexes entre les tables commandes, produits et commerces, permet de récupérer en une seule requête toutes les informations nécessaires à l'affichage d'une commande, réduisant drastiquement le nombre d'appels à la base de données.

7.2.3 Fluidité de l'interface mobile

Le Performance Monitor d'Expo confirme que l'application maintient un taux de rafraîchissement constant de 60 FPS lors du défilement des listes de produits, même avec des images et des données complexes. La consommation mémoire reste stable sans fuite détectée lors de la navigation entre les écrans. L'implémentation du scroll infini avec FlatList, combinée au système de pagination côté serveur (20 produits par page), garantit des temps de chargement rapides : l'écran d'accueil s'affiche en moins d'une seconde, contre 4 à 5 secondes avant l'optimisation.

7.2.4 Gestion des images avec Cloudinary

La migration vers Cloudinary a produit des améliorations mesurables significatives :

- **Temps d'upload moyen** : réduit de 30-60 secondes à 4-6 secondes (amélioration de 83%)
- **Taille de la base de données pour 50 produits** : réduite de 400 Mo à 12 Mo (réduction de 97%)

- **Temps de chargement de l'écran d'accueil sur connexion 3G** : réduit de 10-15 secondes à 2-3 secondes (amélioration de 80%)
- **Optimisation automatique du format** : conversion WebP pour les navigateurs compatibles, compression adaptative selon la qualité de connexion

---

7.3 Qualité du code et maintenabilité

La couverture de tests exhaustive avec 250 tests unitaires (167 frontend + 83 backend) garantit la stabilité et la fiabilité du code. Le taux de réussite de 100% sur l'ensemble des tests valide la conformité de l'application aux spécifications métier. L'organisation modulaire du code, avec une séparation stricte des responsabilités (composants réutilisables, services API centralisés, contextes globaux, styles externalisés), facilite grandement la maintenance et l'évolution future du projet. Le typage strict avec TypeScript réduit les risques d'erreurs à l'exécution et améliore l'expérience de développement grâce à l'autocomplétion et à la documentation intégrée.

7.4 Sécurité et fiabilité

L'application met en œuvre plusieurs mécanismes de sécurité validés par les tests :

- **Authentification robuste** : tokens JWT signés avec secret, expiration automatique après 90 jours, stockage sécurisé via SecureStore, gestion propre de l'expiration avec redirection automatique et nettoyage des données
- **Protection des mots de passe** : hachage bcrypt avec 10 salt rounds, aucun stockage en clair
- **Contrôle d'accès** : middleware d'authentification vérifiant le token sur chaque route protégée, middleware de vérification du type d'utilisateur pour restreindre l'accès aux fonctionnalités vendeur et association
- **Prévention des conditions de course** : transactions PostgreSQL avec verrous FOR UPDATE garantissant l'atomicité des opérations critiques (décrémentation du stock, création de commandes)
- **Protection des secrets** : variables d'environnement externalisées dans .env (exclu du contrôle de version), aucune clé API ou secret codé en dur dans le code
- **Paiement sécurisé** : intégration Stripe avec Payment Intent, aucune manipulation directe des données de carte bancaire côté application

---

7.5 Différenciation par rapport aux solutions existantes

Comme identifié dans l'analyse du marché (section 2), ECO STOCK se distingue des solutions existantes comme Too Good To Go et Phenix sur plusieurs aspects clés désormais implémentés et fonctionnels :

- **Transparence totale** : contrairement au modèle de paniers surprise, l'utilisateur visualise précisément chaque produit avec sa photo, sa description détaillée, son prix, sa quantité et sa date limite de consommation avant d'acheter. Cette transparence favorise une consommation plus réfléchie et réduit les déceptions potentielles.
- **Accompagnement à la consommation** : l'onglet recettes avec son algorithme de matching d'ingrédients et ses suggestions de produits manquants constitue une véritable valeur ajoutée absente des plateformes concurrentes. Cette fonctionnalité accompagne l'utilisateur bien au-delà de l'achat, en l'aidant à valoriser les produits achetés et à éviter le gaspillage à domicile.
- **Dimension sociale visible** : les comptes associations avec accès à des offres réservées ne sont pas une fonctionnalité secondaire cachée, mais un pilier du projet pleinement intégré dans l'architecture et l'interface utilisateur, renforçant l'impact social du projet.
- **Recherche intelligente** : le système de recherche par ingrédients avec tolérance aux fautes (algorithme de Levenshtein) et le filtrage multi-critères offrent une expérience de recherche plus riche et plus flexible que les filtres basiques des concurrents.
- **Sensibilisation intégrée** : les messages écologiques affichés lors des phases de chargement participent à maintenir l'utilisateur conscient des enjeux environnementaux, renforçant l'identité du projet au-delà du simple aspect transactionnel.

7.6 Limites et axes d'amélioration identifiés

Malgré les résultats satisfaisants obtenus, certaines limites et axes d'amélioration ont été identifiés :

- **Recommandations personnalisées** : bien que prévue dans le cahier des charges initial, la fonctionnalité de recommandations basées sur l'historique d'achats et les préférences utilisateur n'a pas été pleinement implémentée. Actuellement, les produits sont triés par proximité géographique ou par date de péremption, mais sans personnalisation avancée. Cette fonctionnalité constitue une évolution prioritaire pour améliorer l'engagement des utilisateurs.
- **Système de notifications** : l'absence de notifications push limite la réactivité des utilisateurs face aux nouvelles offres. L'implémentation de notifications pour alerter d'un nouveau produit proche, d'un changement de statut de commande, ou d'un produit bientôt expiré améliorerait significativement l'expérience utilisateur et favoriserait l'engagement.
- **Partage communautaire** : la possibilité pour les utilisateurs de partager leurs propres recettes ou astuces anti-gaspillage créerait une dimension collaborative absente actuellement, renforçant l'aspect communautaire du projet.

7.7 Bilan global

Le projet ECO STOCK a permis de développer une application mobile fonctionnelle et performante répondant aux objectifs fixés dans le cahier des charges. L'ensemble des fonctionnalités indispensables a été implémenté avec succès, et les résultats des tests de performance démontrent que l'architecture technique choisie est robuste et scalable. La couverture de tests exhaustive et l'organisation modulaire du code garantissent la maintenabilité et l'évolutivité du projet.

La différenciation par rapport aux solutions existantes est bien établie, notamment grâce à la transparence totale sur les produits, à l'accompagnement à la consommation via les recettes, et à l'intégration forte de la dimension sociale avec les comptes associations. Les optimisations apportées (gestion des images avec Cloudinary, pagination avec scroll infini, index de base de données, transactions atomiques) ont permis d'obtenir des performances comparables à celles des applications professionnelles du marché.

Bien que certaines fonctionnalités optionnelles n'aient pas été développées dans le cadre de ce premier livrable (recommandations avancées, notifications, avis), les fondations techniques solides et l'architecture modulaire permettent d'envisager sereinement leur implémentation lors d'itérations futures. Le projet constitue ainsi une base viable et évolutive pour une solution de lutte contre le gaspillage alimentaire différenciée et à fort impact social.

8. Difficultés rencontrées et solutions apportées

Difficulté 1 - Gestion de la concurrence sur le stock lors des achats simultanés

Contexte : Lors des tests avec plusieurs utilisateurs, nous avons découvert un bug critique : deux clients pouvaient acheter simultanément le dernier exemplaire d'un produit. Le premier validait son panier et lançait le paiement. Pendant que le formulaire Stripe s'affichait (quelques secondes), un deuxième utilisateur voyait encore le produit disponible, l'ajoutait au panier et validait également. Résultat : deux paiements réussissaient pour un seul produit en stock.

Cause identifiée : La vérification du stock (SELECT stock FROM produits) et sa décrémentation (UPDATE produits SET stock = stock - 1) étaient deux opérations séparées non atomiques. Entre les deux, une fenêtre temporelle permettait à un autre utilisateur d'effectuer la même vérification avec l'ancien stock, créant une condition de course (race condition).

Solution apportée : Implémentation de transactions PostgreSQL avec verrous exclusifs (FOR UPDATE). Au moment de la création du Payment Intent Stripe, l'API démarre une transaction, sélectionne le produit avec un verrou qui bloque toute autre lecture ou modification, vérifie le stock, puis effectue un commit ou rollback selon la disponibilité. La décrémentation finale du stock se fait après confirmation du paiement, également dans une transaction verrouillée garantissant l'atomicité de l'opération.

Résultat : Les tests avec 10 utilisateurs tentant d'acheter simultanément le dernier produit montrent qu'un seul réussit, les 9 autres recevant immédiatement le message : "Le produit Pain aux céréales n'a plus que 0 unités disponibles". Le problème de sur-vente a été éliminé.

Difficulté 2 - Produits restant visibles après l'heure limite de retrait

Contexte : ECO STOCK fonctionne sur le principe de créneaux de retrait courts (comme Too Good To Go) : un vendeur publie des invendus récupérables uniquement sur une plage horaire précise (exemple : 18h-20h pour un restaurant, 19h30-20h pour une boulangerie). Problème initial : les produits restaient visibles et achetables même après l'heure limite de retrait, créant des situations impossibles à gérer.

Scénario problématique observé : Une boulangerie publie à 18h30 ses 5 baguettes invendues, récupérables entre 19h et 19h30. À 19h45, un client voit encore les baguettes disponibles, les ajoute au panier, valide le paiement Stripe... mais ne peut pas les récupérer car la boulangerie est fermée et l'horaire de retrait est dépassé. Le client est frustré, demande un remboursement, et la boulangerie perd du temps en gestion de litiges.

Solution apportée :

Filtrage automatique dans les requêtes SQL : Modification de toutes les requêtes de listing de produits pour exclure automatiquement les produits hors créneaux en ajoutant les conditions WHERE dlc supérieur ou égal à CURRENT_DATE et pickup_end_time supérieur à CURRENT_TIME. Si un produit a une heure de fin de retrait fixée à 19h30 et qu'il est 19h45, il n'apparaît plus dans les résultats.

Vérification au moment du checkout : Avant de créer le Payment Intent Stripe, vérification explicite pour chaque produit du panier que l'horaire de retrait n'est pas dépassé. Si dépassé, retour d'une erreur 400 avec message explicite indiquant le nom du produit et l'heure limite dépassée.

Affichage du créneau de retrait : Dans les fiches produit, affichage clair du créneau avec l'icône horloge : "Retrait : aujourd'hui de 19h00 à 19h30". Un badge "Bientôt expiré" s'affiche en orange quand il reste moins de 30 minutes avant la fin du créneau.

Script de nettoyage quotidien : Création d'un script cleanupExpiredProducts.js qui s'exécute automatiquement chaque nuit à minuit pour désactiver les produits dont la DLC est dépassée via une requête UPDATE SET is_disponible = false. Les produits ne sont pas supprimés physiquement pour préserver l'historique des commandes passées.

Résultat : Disparition automatique des produits dès que l'heure limite est atteinte. Impossibilité de payer pour un produit hors créneau grâce à la détection au checkout. L'expérience utilisateur est proche de Too Good To Go : les utilisateurs comprennent qu'ils doivent être réactifs pour profiter des offres.

Difficulté 3 - Recherche d'ingrédients avec variations orthographiques et doublons

Contexte : Pour le système de recettes et le tagging des produits, les vendeurs devaient saisir les ingrédients manuellement. Rapidement, nous avons constaté une multiplication des doublons : "tomate", "tomates", "Tomate", "tomate cerise", "tomatte" (faute de frappe), "pâte", "pate", "pates", etc. Cela cassait complètement le matching de recettes : une recette demandant "tomates" ne matchait pas avec un produit taggé "tomate" au singulier.

Problème : La base contenait 347 ingrédients différents dont environ 60% étaient en réalité des doublons. Une recette simple de "pâtes à la tomate" ne matchait avec presque aucun produit car les vendeurs avaient saisi "pate" (sans accent) et "tomate" (singulier) alors que la recette contenait "pâtes" et "tomates".

Solution apportée en plusieurs étapes :

Normalisation automatique : Toute saisie d'ingrédient passe désormais par une fonction de normalisation avant stockage en base de données. Cette fonction convertit le texte en minuscules, supprime tous les accents via la méthode normalize NFD, retire les caractères

spéciaux, et effectue un trim des espaces. Exemple : "Pâtes fraîches" devient "pates fraiches" en base.

Algorithme de fuzzy matching (distance de Levenshtein) : Pour l'autocomplétion vendeur et la recherche de recettes, implémentation d'un algorithme de distance de Levenshtein qui calcule la similarité entre deux chaînes de caractères. Exemples de résultats obtenus : "tomate" et "tomates" ont une distance de 1 caractère donc 85% de similarité (accepté), "ognon" et "oignon" ont une distance de 1 donc 83% de similarité (accepté), "pomme" et "banane" ont une distance de 5 donc 20% de similarité (rejeté). Le seuil de similarité a été fixé à 75% pour accepter une correspondance.

Autocomplétion intelligente : Lors de la saisie vendeur, dès que 2 caractères sont tapés, le système affiche des suggestions d'ingrédients déjà existants dans la base. Si le vendeur commence à taper "tom", le système affiche immédiatement "tomate" et "tomate cerise", évitant ainsi la création d'une nouvelle entrée "tomatte" avec faute de frappe.

Nettoyage manuel de la base : Développement et exécution d'un script de déduplication qui a analysé tous les ingrédients existants, regroupé les variantes similaires, et fusionné les doublons. Les 347 ingrédients initiaux sont passés à 89 ingrédients uniques réellement distincts.

Résultat : Les vendeurs créent beaucoup moins de doublons grâce au système d'autocomplétion intelligent. La qualité des suggestions de recettes proposées aux utilisateurs s'est améliorée.

Difficulté 4 - Lenteur de chargement avec une base de données croissante

Contexte : Lors des tests avec une base de données contenant 500+ produits réalistes, le temps de chargement de l'écran d'accueil atteignait 4 à 5 secondes sur connexion WiFi. Sur connexion 3G, cela montait à 8-10 secondes. L'application semblait "plantée" pendant le chargement initial sans aucun retour visuel. De plus, le scroll dans la liste devenait saccadé et peu fluide sur les appareils Android d'entrée de gamme.

Cause identifiée : L'API retournait tous les produits disponibles en une seule requête HTTP sans aucun système de pagination (parfois 300 à 400 produits en un seul appel). React Native devait alors recevoir et parser un fichier JSON de 2 à 3 mégaoctets, créer 300 à 400 composants ProductCard en mémoire simultanément, et tenter de les rendre tous même ceux complètement hors écran.

Solution apportée :

Pagination côté backend : Implémentation de paramètres de requête page (par défaut 1) et limit (par défaut 20) dans toutes les routes de listing de produits. La requête SQL utilise désormais les clauses LIMIT et OFFSET pour ne retourner qu'un sous-ensemble de résultats. La réponse JSON inclut également des métadonnées : total (nombre total de produits), page (page actuelle), pages (nombre total de pages), et hasMore (booléen indiquant s'il reste des pages à charger).

Scroll infini avec FlatList : Remplacement du composant ScrollView par FlatList de React Native, qui ne rend que les éléments visibles à l'écran plus une petite marge. Configuration de la propriété onEndReached pour détecter quand l'utilisateur approche de la fin de la liste, et onEndReachedThreshold fixé à 0.5 pour anticiper le chargement. Un composant ActivityIndicator s'affiche en bas de liste pendant le chargement de la page suivante.

Optimisations additionnelles : Création d'index PostgreSQL sur les colonnes dlc, category_id et is_disponible qui sont fréquemment utilisées dans les clauses WHERE et ORDER BY. Implémentation du lazy loading des images avec affichage d'un placeholder gris pendant le téléchargement. Mise en cache des catégories qui changent rarement.

Résultat : Le temps de chargement initial de l'écran d'accueil est passé de 4-5 secondes à moins d'1 seconde. Le scroll est maintenant fluide même avec 1000+ produits en base de données. La consommation mémoire a été divisée par 3 sur les appareils Android. L'expérience utilisateur est désormais comparable aux applications professionnelles du marché.

Difficulté 5 - Upload et stockage des images de produits

Contexte : Initialement, les images de produits étaient converties en base64 côté mobile puis stockées directement dans PostgreSQL dans un champ de type TEXT. Cette approche a rapidement montré ses limites avec plusieurs problèmes constatés : upload très lent (15 à 20 secondes pour une photo iPhone de 3-4 mégaoctets), base de données gonflant rapidement (100 produits avec photos représentaient environ 300 mégaoctets), erreur 413 Payload Too Large sur le serveur Node.js, et récupération lente des listes de produits car chaque produit incluait son image base64 complète dans la réponse JSON.

Scénario problématique : Un vendeur prenait une photo de ses produits directement avec son iPhone récent (12 mégapixels, fichier d'environ 4 mégaoctets). L'image convertie en base64 atteignait environ 6 mégaoctets. L'upload saturait complètement la connexion 4G pendant 30 à 40 secondes, et était souvent interrompu par un timeout HTTP avant même la fin de la transmission.

Solution apportée - Migration vers Cloudinary :

Optimisation :

Étape 1 : Sélection de la photo via Expo ImagePicker avec options de compression (quality fixée à 0.7 pour une compression à 70%, largeur maximale de 1080 pixels).

Étape 2 : Conversion en base64 de l'image déjà compressée (taille réduite à 500-800 kilo-octets au lieu de 6 méga-octets).

Étape 3 : Envoi rapide au backend grâce à la compression préalable.

Étape 4 : Le backend utilise le SDK Cloudinary pour uploader l'image avec transformations automatiques (redimensionnement à 800 pixels, qualité auto, format auto pour servir du WebP si supporté).

Étape 5 : Cloudinary retourne une URL sécurisée permanente. Seule cette URL d'environ 50 caractères est stockée dans le champ image_url de la table produits.

Avantages mesurés : Le temps d'upload est passé de 20-30 secondes à 4-6 secondes (5 fois plus rapide). La taille de la base de données pour 50 produits avec photos est passée de 400 mégaoctets à 12 mégaoctets (divisée par 10). Le temps de chargement de l'écran d'accueil sur connexion 3G est passé de 10-15 secondes à 2-3 secondes. Les images sont automatiquement optimisées par Cloudinary (format WebP si le navigateur le supporte, compression adaptative selon la qualité de connexion). Le CDN mondial de Cloudinary sert les images depuis le serveur géographiquement le plus proche de l'utilisateur.

Résultat : L'ajout de produits avec photos est devenu fluide et rapide même sur connexion 4G moyenne. La base de données reste légère et performante. Le chargement des images dans les listes de produits est quasi-instantané grâce au réseau de diffusion de contenu mondial.

Difficulté 6 - Incohérence des données de géolocalisation des commerces

Contexte : Pour afficher les produits triés par proximité géographique, chaque commerce doit disposer de coordonnées GPS précises (latitude et longitude). Dans la première version, les vendeurs saisissaient leur adresse dans un champ texte libre lors de l'inscription. Problème : les vendeurs saisissaient des adresses incomplètes, incorrectes ou ambiguës, rendant la géolocalisation impossible ou complètement fausse.

Exemples d'adresses problématiques observés : "Rue de la République" (sans numéro, sans ville, impossible à géolocaliser précisément), "Boulangerie Paul, Paris" (trop vague car il existe plusieurs dizaines de boulangeries Paul à Paris), "123 Avenue" (adresse incomplète sans nom de voie ni ville), ou encore des coordonnées GPS saisies manuellement mais avec latitude et longitude inversées dans le mauvais ordre.

Conséquences : Certains commerces n'apparaissaient jamais dans les résultats de recherche par proximité car leurs coordonnées étaient nulles ou invalides. D'autres apparaissaient à des emplacements complètement faux, parfois dans un autre pays ou au milieu de l'océan. Les utilisateurs perdaient confiance en voyant un commerce affiché à "2.3 km" alors qu'il était en réalité à 15 kilomètres de distance.

Solution apportée :

Intégration API Nominatim (OpenStreetMap) : Ajout d'un champ d'autocomplétion intelligent sur le formulaire d'inscription vendeur. Dès que le vendeur tape 3 caractères, l'API Nominatim interroge la base OpenStreetMap et suggère une liste d'adresses complètes correspondantes. Exemple : si le vendeur tape "15 rue Vic", le système affiche "15 Rue de la Victoire, 75009 Paris, France"

Extraction automatique des coordonnées GPS : Lorsque le vendeur sélectionne une adresse suggérée dans la liste, l'API Nominatim retourne automatiquement les coordonnées géographiques précises (latitude et longitude avec 7 décimales de précision). Ces

coordonnées sont automatiquement remplies dans les champs latitude et longitude de la table commerces, de manière invisible pour l'utilisateur qui n'a pas à les saisir manuellement.

Validation des coordonnées : Vérification côté serveur que la latitude est bien comprise entre -90 et +90 degrés, et que la longitude est bien comprise entre -180 et +180 degrés. Si une inversion est détectée (exemple : latitude valant 2.28 et longitude valant 48.86 pour un commerce censé être à Paris), le système effectue une correction automatique et affiche une alerte au vendeur.

Résultat : 100% des commerces inscrits sont maintenant correctement géolocalisés avec des coordonnées GPS précises. Le tri par proximité est fiable et pertinent pour les utilisateurs. Le temps moyen d'inscription d'un vendeur a été réduit car il y a beaucoup moins de corrections manuelles nécessaires.

Difficulté 7 - Gestion de l'expiration des sessions JWT sans feedback utilisateur

Contexte : Les tokens JWT (JSON Web Tokens) utilisés pour l'authentification étaient configurés avec une durée d'expiration de 90 jours. Un utilisateur pouvait s'inscrire, utiliser l'application pendant quelques jours, puis ne revenir que 3 mois plus tard. Son token avait expiré entre-temps, mais aucun mécanisme ne gérait proprement cette expiration côté application mobile.

Scénario problématique observé : Un utilisateur ouvre l'application après 95 jours d'inactivité. Il navigue normalement sur l'écran d'accueil qui affiche les produits publics (ne nécessitant pas d'authentification). Quand il essaie d'accéder à son profil utilisateur ou d'ajouter un produit au panier, l'API retourne une erreur 401 Unauthorized. Côté mobile, aucun traitement n'était prévu : l'écran restait figé sur un spinner de chargement infini sans aucun message d'erreur. L'utilisateur pensait que l'application était cassée ou bloquée et finissait par la désinstaller par frustration.

Causes identifiées : Aucune détection de l'erreur 401 dans le service API côté mobile. Pas de nettoyage du token expiré stocké dans SecureStore. Pas de redirection automatique vers l'écran de connexion. Aucun message explicatif affiché à l'utilisateur pour comprendre ce qui se passe.

Solution apportée :

Intercepteur global d'erreurs API : Création d'un intercepteur dans le service API qui détecte automatiquement toutes les réponses HTTP avec code 401. Lorsqu'une erreur 401 est détectée, le système nettoie immédiatement le token expiré du SecureStore, affiche une alerte explicite avec le message "Session expirée. Veuillez vous reconnecter.", et redirige automatiquement l'utilisateur vers l'écran de connexion.

Vérification proactive au lancement de l'application : Au démarrage de l'application, le token JWT stocké est décodé (sans nécessiter d'appel API) pour vérifier sa date d'expiration. Si le token est déjà expiré, le nettoyage et la redirection sont effectués immédiatement. Si le

token expire dans moins de 7 jours, un message d'avertissement est affiché : "Votre session expire bientôt, pensez à vous reconnecter pour ne pas perdre vos données".

Résultat : Expérience utilisateur claire avec message explicite au lieu d'un écran figé incompréhensible. Aucune session zombie (tokens expirés sont systématiquement nettoyés automatiquement).

Difficulté 8 - Dépassement de la limite de payload lors de l'upload de photos haute résolution

Contexte : Les smartphones récents (iPhone 13 et versions ultérieures, Samsung Galaxy S21 et versions ultérieures) capturent des photos en très haute résolution (entre 12 et 48 mégapixels, produisant des fichiers de 4 à 8 mégaoctets par photo). Convertir ces images en base64 pour les envoyer à l'API créait des requêtes HTTP de 6 à 12 mégaoctets.

Problèmes rencontrés :

Problème 1 - Timeout et erreur serveur : Le serveur Express avait une limite de taille de payload configurée par défaut à 10 mégaoctets. Les photos dépassant cette limite déclenchaient systématiquement une erreur 413 Payload Too Large côté serveur. Même en augmentant la limite serveur à 50 mégaoctets, les uploads prenaient entre 30 et 60 secondes sur connexion mobile 4G, très souvent interrompus par des timeouts HTTP avant la fin de la transmission.

Problème 2 - Base de données gonflée : Les images stockées en base64 dans PostgreSQL alourdissaient dramatiquement la taille de la base de données. Après l'ajout de seulement 50 produits avec leurs photos, la taille totale de la base de données atteignait déjà 400 mégaoctets, ralentissant toutes les requêtes et augmentant les coûts d'hébergement.

Problème 3 - Chargement lent des listes : Chaque requête pour récupérer la liste des produits retournait les images complètes encodées en base64 dans la réponse JSON, créant des fichiers de plusieurs mégaoctets à télécharger. Sur connexion 3G, le simple chargement de l'écran d'accueil prenait 10 à 15 secondes.

Solution apportée :

Compression côté mobile avant envoi : Configuration d'Expo ImagePicker pour compresser automatiquement les images sélectionnées avec une qualité fixée à 0.7 , une largeur maximale limitée à 1080 pixels, et activation de la sortie base64. Résultat immédiat : une photo iPhone de 4 mégaoctets est automatiquement réduite à 600-800 kilo-octets avant même l'envoi.

Upload vers Cloudinary depuis le backend : Le backend reçoit l'image compressée en base64, puis utilise le SDK Cloudinary pour uploader l'image vers leur service cloud avec des transformations automatiques (largeur maximale 800 pixels avec crop limit, qualité auto, format auto pour servir du WebP si le client le supporte). Cloudinary retourne une URL sécurisée permanente. Seule cette URL d'environ 50 caractères est stockée dans le champ image_url de la table produits.

Comparaison des métriques avant et après :

Temps d'upload moyen : avant 30-60 secondes, après 4-6 secondes. Taille base de données pour 50 produits : avant 400 mégaoctets, après 12 mégaoctets. Temps chargement écran accueil sur 3G : avant 10-15 secondes, après 2-3 secondes. Limite de payload serveur atteinte : avant oui fréquemment, après non jamais. Optimisation automatique du format (WebP) : avant non, après oui automatique.

Résultat : L'ajout de produits avec photos est devenu fluide et rapide même sur connexion 4G moyenne. La base de données reste légère et performante même avec des centaines de produits. Les images sont servies depuis le réseau de diffusion de contenu mondial de Cloudinary, optimisées automatiquement selon le type d'appareil de l'utilisateur final.

9. Conclusion et perspectives

Le projet ECO STOCK visait à créer une application mobile pour réduire le gaspillage alimentaire en connectant commerçants et consommateurs autour des invendus. Après plusieurs mois de développement, nous pouvons affirmer que les objectifs ont été atteints et que l'application fonctionne conformément à nos attentes.

9.1 Bilan du projet

Toutes les fonctionnalités essentielles du cahier des charges ont été implémentées. Les vendeurs peuvent publier leurs invendus avec photos, prix, DLC et horaires de retrait. Les clients recherchent et achètent des produits en toute transparence, avec des filtres performants et un paiement sécurisé via Stripe. Le système de recettes propose des idées concrètes pour utiliser les produits achetés, ce qui va au-delà d'une simple marketplace.

La dimension sociale est bien présente avec les comptes associations qui accèdent à des offres réservées. La géolocalisation fonctionne et propose des temps de trajet écologiques (à pied, vélo, transport). Les tests de performance montrent que l'application tient la charge, 912 requêtes par seconde côté serveur, 60 FPS constant sur mobile, chargement en moins d'une seconde.

9.2 Ce qu'on a appris

Ce projet nous a fait progresser sur plusieurs aspects. Techniquement, on a découvert React Native et TypeScript pour le mobile, Node.js et PostgreSQL pour le backend. On a dû gérer des problèmes concrets comme la concurrence sur le stock (résolu avec des transactions PostgreSQL), l'upload d'images (migration vers Cloudinary), ou la recherche intelligente avec tolérance aux fautes.

La méthode Scrum nous a vraiment aidés à nous organiser. Les dailys, les sprints de deux semaines et les rétros avec nos encadrants nous ont permis d'avancer régulièrement et de corriger le tir quand nécessaire. Travailler tous ensemble sur l'ensemble du projet plutôt que de se séparer par domaine était le bon choix : chacun a pu toucher au front, au back et à la base de données.

Les 250 tests unitaires qu'on a écrits (100% de réussite) nous ont sauvé plusieurs fois en détectant des bugs avant qu'ils deviennent bloquants. C'est une pratique qu'on gardera pour nos futurs projets.

9.3 Les difficultés rencontrées

On a eu notre lot de galères. La gestion du stock quand deux personnes achètent le dernier produit en même temps a nécessité de comprendre les verrous PostgreSQL. Les images qui saturaient la base de données nous ont forcés à repenser complètement notre approche. La

recherche d'ingrédients qui ne fonctionnait pas à cause des fautes de frappe et des pluriels a demandé l'implémentation d'un algorithme de distance de Levenshtein.

Mais chaque problème résolu nous a appris quelque chose. On sait maintenant gérer des transactions atomiques, optimiser des uploads d'images, ou faire de la recherche floue. Ces compétences sont directement réutilisables en entreprise.

9.4 Ce qui pourrait être amélioré

Plusieurs pistes nous semblent intéressantes pour aller plus loin :

Recommandations personnalisées : actuellement, les produits sont triés par proximité ou par DLC, mais on pourrait analyser l'historique d'achats pour proposer des suggestions vraiment adaptées à chaque utilisateur.

Notifications push : pour alerter d'un nouveau produit proche qui correspond aux préférences, d'un changement de statut de commande, ou d'une offre qui expire bientôt.

Statistiques d'impact : afficher le CO2 économisé, le nombre de repas sauvés et les économies réalisées valoriserait l'engagement écologique de chacun.

Partage communautaire : laisser les utilisateurs partager leurs propres recettes anti-gaspi créerait une vraie communauté autour de l'app.

9.5 Mot de la fin

Au final, ECO STOCK remplit son rôle, une application fonctionnelle qui répond au problème du gaspillage alimentaire tout en permettant d'acheter moins cher. On a réussi à se démarquer des solutions existantes avec la transparence sur les produits, le système de recettes et l'intégration des associations.

Les choix techniques se sont corrects. L'architecture est bonne, le code est maintenable, et les performances sont excellentes. On a maintenant une base solide qu'on pourrait faire évoluer si on le souhaitait.

Ce qui ressort surtout, c'est qu'on a beaucoup appris. Sur le développement mobile et backend évidemment, mais aussi sur la gestion de projet, le travail d'équipe et la résolution de problèmes. Le fait que le projet ait un impact potentiel sur le gaspillage alimentaire lui donne un sens qui va au-delà de l'exercice scolaire.

On termine ce projet avec une application qui fonctionne, des compétences techniques renforcées, et une meilleure compréhension de ce qu'implique un développement complet. C'est une bonne expérience à ajouter au portfolio.

Synthèses individuelles

Limonta Evan :

Tout au long du projet ECO STOCK, j'ai surtout travaillé sur le backend Node.js/Express. J'ai aussi fait du frontend React Native pour bien séparer les styles du code métier, un principe qu'on a appliqué partout dans l'application.

Ce qui m'a le plus intéressé, c'est l'implémentation des algorithmes mathématiques comme Haversine pour calculer les distances et Levenshtein pour la recherche tolérante aux fautes. Il fallait bien comprendre les formules pour les coder correctement. Côté optimisation, j'ai mis en place la pagination avec scroll infini qui a divisé le temps de chargement par 4, passant de 4-5 secondes à moins d'une seconde. Satisfait aussi d'avoir pu faire des API REST en respectant le principe REST.

Le plus gros challenge a été la gestion de la concurrence sur les stocks. J'ai dû utiliser des transactions PostgreSQL avec FOR UPDATE pour éviter que deux personnes achètent le même dernier produit en même temps. La migration vers Cloudinary pour les images a aussi demandé pas mal de boulot, mais ça a réduit la taille de la base et amélioré le temps d'upload.

J'ai vraiment apprécié que le projet soit notre idée. On a pu faire notre propre cahier des charges au début, pas juste du développement. Le rendu est très satisfaisant vu le résultat très positif des performances de l'application.

Bouzidi Rayen: J'ai bien aimé ce projet car c'est la première fois que nous avions une certaine liberté dans le choix du sujet, cela m'a encore plus motivé à travailler car c'est une idée que nous avons trouvée avec mes camarades et pas un sujet imposé. Également nous avons pu travailler sur un aspect plus complet, lors des différents projets nous nous occupions du développement mais là il y a eu une partie cadrage et rédaction du cahier des charges au début, ce qui a permis d'avoir un aspect beaucoup plus global du projet. J'ai également pu développer plusieurs compétences, notamment en termes de développement car je n'avais jamais codé en React Native. J'ai donc tout appris grâce à ce projet, mais j'ai pu également améliorer mes compétences dans les divers domaines de l'application, comme la création de maquette ou le back end.

NGUYEN Phuong :
J'ai particulièrement apprécié ce projet car il s'agit d'un projet libre, ce qui nous a permis de concevoir l'application en entier avec pour seule contrainte avoir une dimension éthique et écologique. Cette liberté nous a offert la possibilité de réfléchir en profondeur aux besoins réels des utilisateurs, de définir un cahier des charges cohérent et de proposer des fonctionnalités pertinentes, tout en donnant du sens au développement de l'application. Pour la réalisation de ce projet, il a fallu tout d'abord analyser les besoins principaux et identifier des fonctionnalités absentes ou peu développées chez les autres concurrents pour nous

démarquer et proposer une application plus complète. J’ai pu développer mes compétences notamment la création de maquette Figma, le développement mobile en React Native que j’avais déjà codé dans ma précédente SAE et j’ai pu approfondir ma maîtrise de ces outils et améliorer la structuration de mon code. J’ai également pu acquérir des nouvelles compétences telles que l’intégration d’une API REST, l’intégration de l’API de paiement Stripe et l’utilisation du back end PostgreSQL que je n’avais jamais pratiqué auparavant. Je suis très satisfait du résultat de notre projet ainsi du travail réalisé par l’ensemble de l’équipe. Ce projet m’a permis de consolider mes compétences techniques et de mieux appréhender le développement d’une application mobile complète dans un contexte de travail en équipe. Ce projet constitue une expérience très enrichissante pour la réalisation de futurs projets académiques ainsi que dans le cadre de stages, en me permettant de développer davantage mes compétences.

ATCHEKZAI Ismail :
Dans le cadre du projet ECO STOCK mon intervention s'est principalement focalisée sur la cohérence visuelle et l'expérience utilisateur. J'ai travaillé sur l'élaboration des maquettes via Figma pour définir l'identité graphique de l'application avant de basculer sur le développement Frontend en React Native.
Ce projet a été pour moi une bonne expérience notamment grâce à la liberté qui nous a été laissée dans le choix du sujet. Travailler sur une problématique actuelle comme le gaspillage alimentaire a donné un véritable sens à notre travail, je n'avais pas l'impression de coder un simple exercice scolaire mais de concevoir un produit qui pourrait avoir une utilité réelle. J'ai également beaucoup apprécié la dynamique de notre groupe. L'utilisation de la méthode Scrum nous a permis de rester organisés et de voir le projet avancer concrètement à chaque sprint ce qui était très motivant.

Bibliographie / Webographie

- ADEME – Lutte contre le gaspillage alimentaire

  https://agirpourlatransition.ademe.fr/particuliers/reduire-dechets/eviter-gaspillage/reduire-gaspillage-alimentaire

- Too Good To Go – Présentation du service
  https://www.toogoodtogo.com

- Phenix – Présentation de la plateforme
  https://www.wearephenix.com

- Documentation React Native
  https://reactnative.dev

- Documentation Node.js
  https://nodejs.org

- Documentation Express.js
  https://expressjs.com

- Documentation PostgreSQL
  https://www.postgresql.org/docs

- Documentation Stripe – Paiements en ligne
  https://stripe.com/docs

- Documentation Cloudinary – Gestion des images
  https://cloudinary.com/documentation

- JSON Web Token (JWT) – Introduction
  https://jwt.io/introduction